VECTOR

MICROSAR CAN Driver

Technical Reference

NXP Semiconductors

ARM32

FlexCAN3

Version 1.06.00

Authors	Robert Schelkle, Roman Linder, Benjamin Schütterle, Yacine Ouldboukhitine
Status	Released

1 Document Information

1.1 History

Author	Date	Version	Remarks
Robert Schelkle	2016-08-29	1.00.00	Initial creation
Roman Linder	2016-12-21	1.01.00	S32K support
Roman Linder	2017-04-13	1.02.00	MWCT101xS support
Benjamin Schütterle	2018-03-12	1.03.00	Extended RAM Check support for S32K
Roman Linder	2018-03-23	1.04.00	IMX 8QuadXPlus/IMX 8DualXPlus support
Benjamin Schütterle	2018-04-11	1.05.00	GNU Compiler support and update API Description S32K 116/118 support
Yacine Ouldboukhitine	2018-10-31	1.06.00	Support of Virtual Addressing Support of Silent Mode Support SafeBsw for S32K1xx

Table 1-1 History of the Document

1.2 Reference Documents

No.	Title	Version
[1]	AUTOSAR_SWS_CAN_DRIVER.pdf	2.4.6 + 3.0.0 + 4.0.0
[2]	AUTOSAR_BasicSoftwareModules.pdf	V1.0.0
[3]	AUTOSAR_SWS BSW Scheduler	V1.1.0
[4]	AUTOSAR_SWS_CAN_Interface.pdf	3.2.7 + 4.0.0 + 5.0.0
[5]	AN-ISC-8-1118 MICROSAR BSW Compatibility Check	V1.0.0

Table 1-2 Reference Documents

1.3 Scope of the Document

This document describes the functionality, API and configuration of the MICROSAR CAN driver as specified in [1]. The CAN driver is a hardware abstraction layer with a standardized interface to the CAN Interface layer.

Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

Illustrations

Tables

2 Hardware Overview

The following table summarizes information about the CAN Driver. It gives you detailed information about the derivatives and compilers. As very important information the documentations of the hardware manufacturers are listed. The CAN Driver is based upon these documents in the given version.

Derivative	Compiler	Hardware Manufacturer Document	Version
IMX6S IMX6U IMX6D IMX6Q IMX6X S=Solo U=DualLite D=Dual Q=Quad X=SoloX	GreenHills IAR GNU	i.MX 6Dual/6Quad Multimedia Applications Processor Reference Manual. i.MX 6Solo/6DualLite Applications Processor Reference Manual. i.MX 6SoloX Applications Processor Reference Manual	Rev. B, 07/2011 Rev. 1, 04/2013 Rev A, 02/2014
IMX8DXP IMX8QXP D=Dual Q=Quad P=Plus	GreenHills IAR GNU	i.MX 8DualXPlus/8QuadXPlus Applications Processor Reference Manual	Rev. A, 6/2017
S32K116 S32K118 S32K142 S32K144 S32K146 S32K148	GreenHills IAR GNU	S32K1xx Series Reference Manual	Rev. 6, 12/2017
MWCT1013S MWCT1014S	GreenHills IAR GNU	WCT101xS Series Reference Manual	Rev. 0, 10/2016

Table 2-1 Supported Hardware Overview

Derivative: This can be a single information or a list of derivatives, the CAN Driver can be used on.
Compiler: List of Compilers the CAN Driver is working with
Hardware Manufacturer Document Name: List of hardware documentation the CAN Driver is based on.
Version: To be able to reference to this hardware documentation its version is very important.

2.1 Hardware Software Interface

The CAN driver is based upon the hardware manufacturer documents given in Table 2-1 and is applicable for the FlexCAN.

A detailed description on how to access the FlexCAN controller is given in section 4.9 ‘Hardware Specific’.

3 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CAN as specified in [1].

Since each hardware platform has its own behavior based on the CAN specifications, the main goal of the CAN driver is to give a standardized interface to support communication over the CAN bus for each platform in the same way. The CAN driver works closely together with the higher layer CAN interface.

Supported AUTOSAR Release*:	3 and 4	
Supported Configuration Variants: **(Supported AUTOSAR Standard Conform Features)**	Pre-Compile Link-Time Post-Build Loadable Post-Build Selectable (MICROSAR Identity Manager)	
Vendor ID:	CAN_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	CAN_MODULE_ID	80 decimal (according to ref. [2])
AR Version:	CAN_AR_RELEASE_MAJOR_VERSION CAN_AR_RELEASE_MINOR_VERSION CAN_AR_RELEASE_REVISION_VERSION	AUTOSAR Release version (BCD coded)
SW Version:	CAN_SW_MAJOR_VERSION CAN_SW_MINOR_VERSION CAN_SW_PATCH_VERSION	MICROSAR Can module version (BCD coded)

* For the precise AUTOSAR Release 3.x and 4.x please see the release specific documentation.

3.1 Architecture Overview

The following figure shows where the CAN is located in the AUTOSAR architecture.

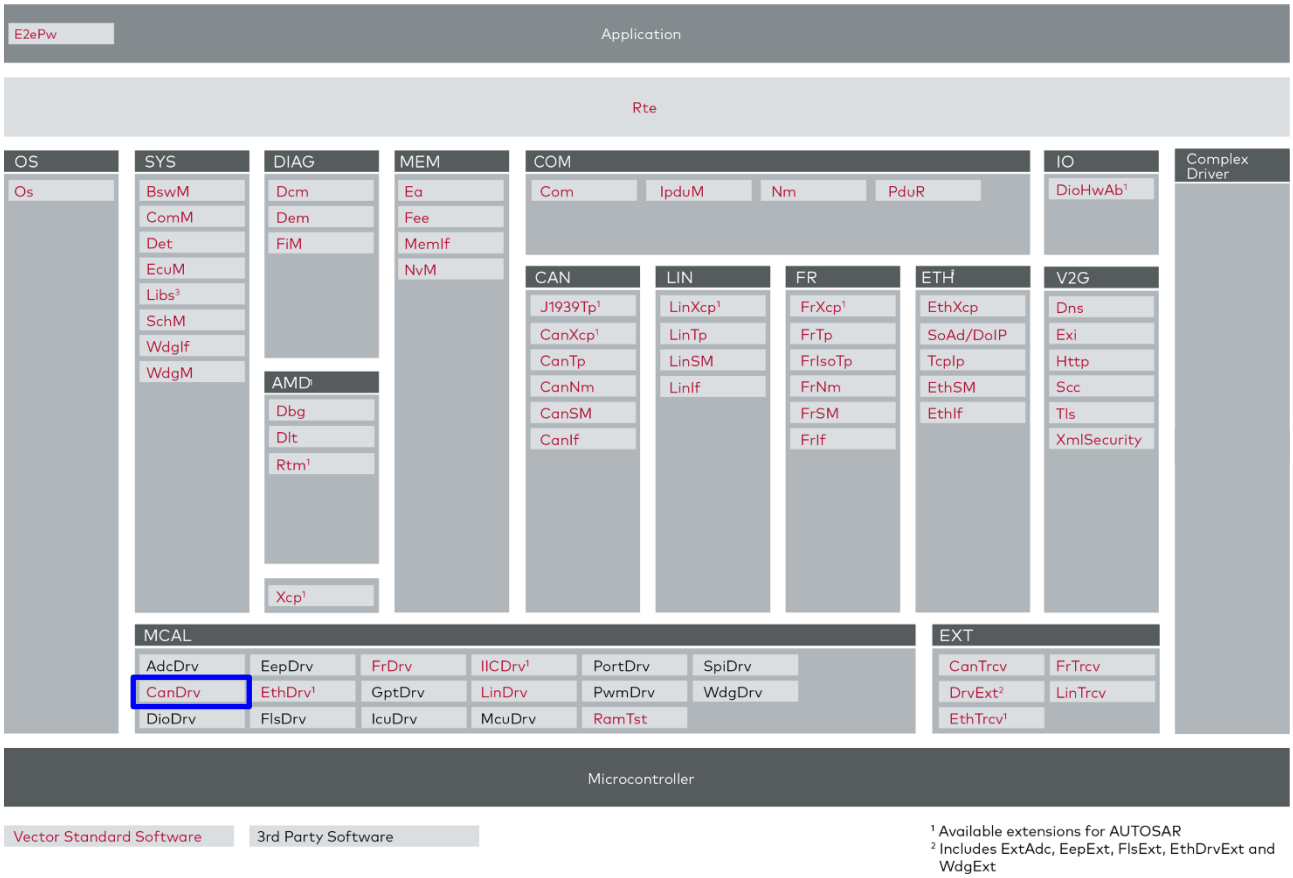


Figure 3-1 AUTOSAR 3.x Architecture Overview

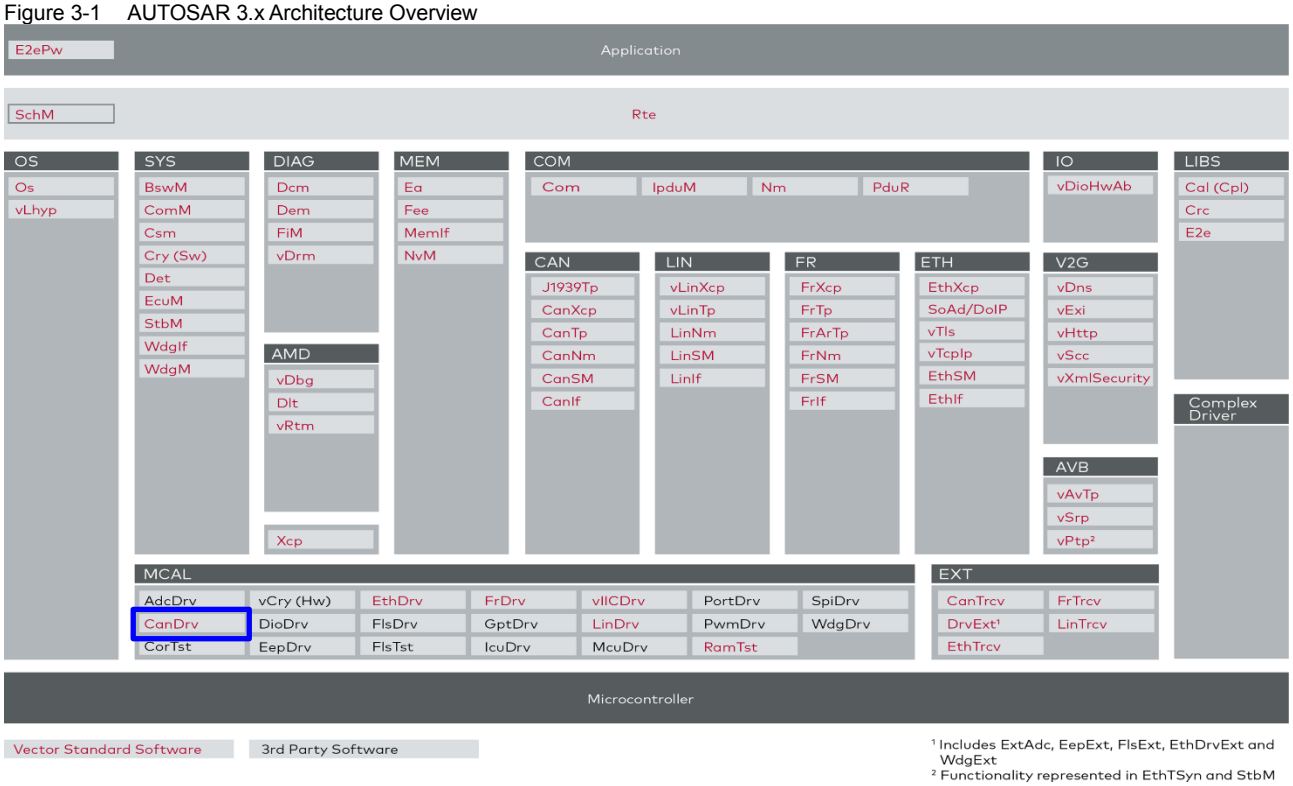


Figure 3-2 AUTOSAR architecture

The next figure shows the interfaces to adjacent modules of the CAN. These interfaces are described in chapter 6.

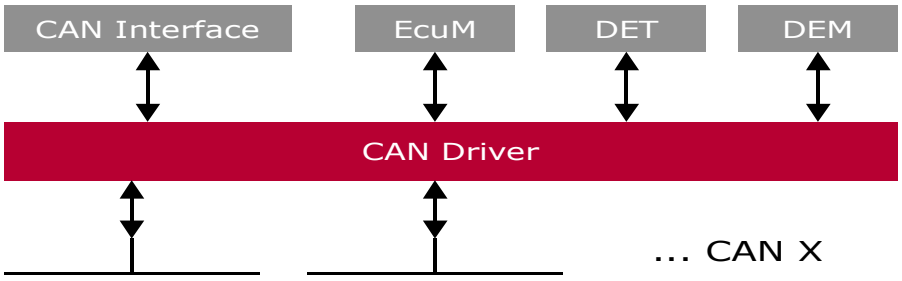


Figure 3-3 Interfaces to adjacent modules of the CAN

4 Functional Description

4.1 Features

The features listed in this chapter cover the complete functionality specified in [1].

The "supported" and "not supported" features are presented in the following table. For further information of not supported features also see chapter 8.

Feature Naming	Short Description	GENy	CFG5
Initialization			
Driver	General driver initialization function Can_Init()	■	■
Controller	Controller specific initialization function Can_InitController().	■	■
Communication			
Transmission	Transmitting CAN frames.	■	■
Transmit confirmation	Callback for successful Transmission.	■	■
Reception	Receiving CAN frames.	■	■
Receive indication	Callback for receiving Frame.	■	■
Controller Modes			
Sleep mode	Controller support sleep mode (power saving).	□	□
Wakeup over CAN	Controller support wakeup over CAN.	□	□
Stop mode	Controller support stop mode (passive to CAN bus).	■	■
Bus Off detection	Callback for Bus Off event.	■	■
Silent mode	Support SilentMode where the controller only listen passive.	□	■ (see 4.4.5)
Polling Modes			
Tx confirmation	Support polling mode for Transmit confirmation.	■	■
Reception	Support polling mode for Reception.	■	■
Wakeup	Support polling mode for Wakeup event.	□	□
Bus Off	Support polling mode for Bus Off event.	■	■
Mode	MICROSAR4x only: Support polling mode for mode transition.	■	■
Mailbox objects			
Tx BasicCAN	Standard mailbox to send CAN frames (Used by CAN Interface data queue).	■	■

Multiplexed Tx	Using 3 mailboxes for Tx BasicCAN mailbox (external priority inversion avoided).	■	■
Tx FullCAN	Separate mailbox for special Tx message used.	■	■
Maximum amount	Available amount of mailboxes.	see section 4.3.1	see section 4.3.1
Rx FullCAN	Separate mailbox for special Rx message used.	■	■
Maximum amount	Available amount of mailboxes.	see section 4.3.1	see section 4.3.1
Rx BasicCAN	Standard mailbox to receive CAN frame (depending on hardware, FIFO or shadow buffer supported).	■	■
Maximum amount	Available amount of BasicCAN objects.	see section 4.3.1	see section 4.3.1
Others			
DEM	Support Diagnostic Event Manager (error notification).	■	□***
DET	Support Development Error Detection (error notification).	■	■
Version API	API to read out component version.	■	■
Maximum supported controllers	Maximum amount of supported controllers (hardware channels).	8	8
Cancellation of Tx objects	Support of Tx Cancellation (out of hardware). Avoid internal priority inversion.	■	■
Identical ID cancellation	Tx Cancellation also for identical IDs.	■	■
Standard ID types	Standard Identifier supported (Tx and Rx).	■	■
Extended ID types	Extended Identifier supported (Tx and Rx).	■	■
Mixed ID types	Standard and Extended Identifier supported (Tx and Rx).	■	■
CAN FD Mode1	FD frames with baudrate switch supported (Tx and Rx).	□	■
CAN FD Mode2	FD frames up to 64 data bytes supported (Tx and Rx).	□****	■
Hardware Loop Check (Timeout monitoring)	To avoid possible endless loops (occur by hardware issue).	■	■
AutoSar extensions			
Individual Polling	Support individual polling mode (selectable for each mailbox separate).	■*	■*
Multiple Rx Basic CAN	Support Multiple Rx BasicCAN objects. This gives the possibility to use multiple Filters and optimize acceptance Filtering as well as avoid overrun.	see section 4.3.1*	see section 4.3.1*

Multiple Tx Basic CAN	Support Multiple Tx BasicCAN objects. Used to send different Tx groups over separate mailboxes with different buffering behavior (see CanInterface).	□*	■*
Rx Queue	Support Rx Queue. This give the possibility to buffer received data in interrupt context but handle it asynchronous in polling task.	■*	■*
Secure Rx Buffer used	Special hardware buffer used to temporary save received data.	□	□
Hardware Loop Check by Application	“Hardware Loop Check” can be defined to be done by application (special API available)	■	■
Configurable “Nested CAN Interrupts”	Nested CAN interrupts allowed, and can be also switched to none-nested.	□	□
Report CAN_E_TIMEOUT DEM as DET	Report CAN_E_TIMEOUT (Hardware Loop Check / Timeout monitoring) to DET instead of DEM.	■	■
Support Mixed ID	Force CAN driver to handle Mixed ID (standard and extended ID) at pre-compile-time to expand the ID type later on.	■	■
Optimize for one controller	Activate this for 1 controller systems when you never will expand to multi-controller. So that the CAN driver works more efficient	■	■
Dynamic FullCAN Tx ID (***)	Always write FullCAN Tx ID within CanWrite() API function. Deactivate this to optimize code when you do not use FullCAN Tx objects dynamically.	■	□
Size of Hw HandleType	Support 8bit or 16bit Hardware Handles depend on hardware usage.	■	■
Generic PreCopy	Support a callback function for receiving any CAN message (following callbacks could be suppressed)	■	■
Generic Confirmation	Support a callback function for successful transmission of any CAN message (following callbacks could be suppressed)	■	■
Get Hardware Status	Support a API to get hardware status Information (see Can_GetStatus())	■	■
Interrupt Category selection	Support Category 1 or Category 2 Interrupt Service Routines for OS	■	■
Common CAN	Support merge of 2 controllers in hardware to get more Rx FullCAN objects	□**	□**
Overrun Notification	Support DET or Application notification cause by overrun (overwrite) of an Rx message (BasicCAN and FullCAN)	■ (see 4.8.1.2)	■ (see 4.8.1.2)

RAM check	Support CAN mailbox RAM check	■	■
Extended RAM Check	Support extended RAM check. Handling of individual deactivated mailboxes and controllers.	□	■*****
Multiple ECU configurations (***)	The feature Multiple ECU is usually used for nodes that exist more than once in a car. At power up the application decides which node should be realized.	■	□
Generic PreTransmit	Support a callback function with pointer to Data, right before this data will be written in Hardware mailbox buffer to send. (Use this to change data or cancel transmission)	■	■

Table 4-1 Supported features

■ Feature is supported

□ Feature is not supported

* HighEnd Licence only

** Project specific (may not be available)

*** Not supported or cannot be configured for MicroSar4

**** Only available for MicroSar403

***** Only available for S32K and MWCT

4.2 Initialization

`Can_Init()` has to be called to initialize the CAN driver at power on and sets controller independent init values. This function has to be called before `Can_InitController()`.

MicroSar3 only: Use `Can_InitStruct()` to change the used baud rate and filter settings like given in the Initialization structure from the Tool. The used default set by `Can_InitMemory()` is the first structure. This API has to be called before `Can_InitController()` but after `Can_InitMemory()`.

MICROSAR401 only: baud rate settings given by Can_InitController parameter.

`Can_InitController()` initializes the controller, given as parameter, and can also be used to reinitialize. After this call the controller stays in stop-mode until the CAN Interface changes to start-mode.

`Can_InitMemory()` is an additional service function to reinitialize the memory to bring the driver back to a pre-power-on state (not initialized). Afterwards `Can_Init()` and `Can_InitController()` have to be called again. It is recommended to use this function before calling `Can_Init()` to secure that no startup-code specific pre-initialized variables affect the driver startup behavior.

4.3 Communication

`Can_Write()` is used to send a message over the mailbox object given as "Hth". The data, DLC and ID is copied into the hardware mailbox object and a send request is set. After sending the message the CAN Interface `CanIf_TxConfirmation()` function is called. Right before the data is copied in mailbox buffer the ID, DLC and data may be changed by `Appl_GenericPreTransmit()` callback.

When “Generic Confirmation“ is activated the callback `Appl_GenericConfirmation`() will be called before `CanIf_TxConfirmation()` and the call to this can be suppressed by `Appl_GenericConfirmation`() return value.

For Tx messages the ID will be copied. (Exception: feature “Dynamic FullCAN Tx ID” is deactivated, then the FullCAN Tx messages will be only set while initialization)

If the mailbox is currently sending the status busy will be returned. Then the message may be queued in the CAN interface (if feature is active).

If cancellation in hardware is supported the lowest priority ID inside currently sending object is canceled, and therefore re-queued in the CAN Interface.

`Appl_GenericPreCopy`() (if activated) is called and depend on return value also `CanIf_RxIndication()` as a CAN Interface callback, is called when a message is received. The receive information like ID, DLC and data are given as parameter.

When Rx Queue is activated the received messages (polling or interrupt context) will be queued (same queue over all channels). The Rx Queue will be read by calling `Can_Mainfunction_Read` () and the Rx Indication (like `CanIf_RxIndication`()) will be called out of this context. Rx Queue is used for Interrupt systems to keep Interrupt latency time short.

4.3.1 Mailbox Layout

The FlexCAN3 Controller that supports CAN-FD has the limitation that the hardware reception Fifo (RxFifo) cannot be used in combination with the CAN-FD feature. Due to this hardware characteristic the CAN driver does support two different mailbox layouts: the RxFifo mailbox layout and the classic mailbox layout. For CAN-FD (Mode1 or Mode2) configurations the classic mailbox layout is used for all channels. If the CAN-FD feature is deactivated in the configuration tool the RxFifo layout is used for all channels.

The generation tool supports a flexible allocation of mailboxes. In the following sections the possible mailbox layouts are shown.

4.3.1.1 RxFifo Mailbox Layout

Please note
If the RxFifo mailbox layout is used the CAN driver supports one Rx BasicCAN object. The Rx BasicCAN object is represented by the RxFifo of the CAN controller. The feature “Multiple BasicCAN” is not supported.

Object Number	Object Type	Amount of Objects	Comment
Position: 0 Occupation: 0 ... a	Rx BasicCAN	8 … 38	For the BasicCAN object the RxFifo is used. All CAN messages, except for the FullCAN messages, are received via the BasicCAN message object. The amount of used objects for the RxFifo depends on the used reception filters that are configured in the generation tool.
a+1 … b	Reserved	0 … 1	If the workaround for ERR005829 is enabled this mailbox is reserved. See 8.2 for detailed information.
b+1 … c	Rx FullCAN	0 … n_{max}-9	These objects are used to receive specific CAN messages. The user defines statically (Generation Tool) that a CAN message should be received in a FullCAN message object. The Generation Tool distributes the message to the FullCAN objects.
c+1 …d	Tx BasicCAN	X * Y	These objects are used to send several messages. If the transmit message object is busy, the transmit request is stored in a CAN Interface queue (if activated). X = amount of TxBasicCANs, Y = 1 or 3 depend on Multiplexed Tx feature.
d+1 … e	Tx FullCAN	0 ... n_{max}-9	These objects are used to send a certain message. The user must define statically (Generation Tool) which CAN messages are located in such Tx FullCAN objects. The Generation Tool distributes the messages to the FullCAN objects.
e+1 … n_{max}-1	Unused	0 ... n_{max}-9	These objects are not used. It depends on the configuration of receive and transmit objects if unused objects are available.

Table 4-2 RxFifo mailbox layout

Note
n_{max} is the maximum amount of available mailboxes per CAN Controller and depends on the used derivative.

By default the RxFifo occupies 8 message buffers (index 0 to 7 in the message buffer hardware layout) to store up to 6 messages and 8 Acceptance Filters. The user can add sets of 8 filters by the generation tool. Every set of filters occupies two more message buffers. The maximum amount of filters is 128, so maximal 38 message buffers get occupied by the RxFifo. For further information see section 4.9.2 or hardware manual of semiconductor manufacturer.

The “CanObjectId” (ECUc parameter) numbering is done in following order: FullCAN Tx, BasicCAN Tx, Unused, FullCAN Rx, BasicCAN Rx (like shown above). “CanObjectId’s” for

next controller begin at end of last controller. Gaps in “CanObjectId” for unused mailboxes may occur.

4.3.1.2 Classic Mailbox Layout

Please note
In the classic mailbox layout the RxFifo is not used. As a consequence the Rx BasicCAN objects are represented by a set of message buffers. The composition of Rx BasicCAN objects are described in the column “Comment” of the table below. The feature Multiple BasicCAN is supported in classic mailbox layout configurations.

Object Number	Object Type	Amount of Objects	Comment
0	Reserved	0 … 1	If the workaround for ERR005829 is enabled this mailbox is reserved. See 8.2 for detailed information.
0/1 … a	Rx FullCAN	0 … n_{max}-3	These objects are used to receive specific CAN messages. The user defines statically (Generation Tool) that a CAN message should be received in a FullCAN message object. The Generation Tool distributes the message to the FullCAN objects.
a+1 … b	Rx BasicCAN	2 … n_{max}-3	All other CAN messages are received via the Rx BasicCAN objects. By default there is one Rx BasicCAN object available. If the feature ‘Multiple BasicCAN Objects’ (High-End feature) is enabled the number of used Rx BasicCAN objects can be configured. Every Rx BasicCAN object consists of two hardware mailboxes (shadow buffer mechanism). Mixed ID systems without Multiple BasicCAN feature: In this special case there are two Rx BasicCAN objects allocated. One to receive standard ID frames and the other to receive extended ID messages.
b+1 …c	Tx BasicCAN	X * Y	These objects are used to send several messages. If the transmit message object is busy, the transmit request is stored in a CAN Interface queue (if activated). X = amount of TxBasicCANs, Y = 1 or 3 depend on Multiplexed Tx feature.
c+1 … d	Tx FullCAN	0 ... n_{max}-3	These objects are used to send a certain message. The user must define statically (Generation Tool) which CAN messages are located in such Tx FullCAN objects. The Generation Tool distributes the messages to the FullCAN objects.
d+1 … n_{max}-1	Unused	0 ... n_{max}-3	These objects are not used. It depends on the configuration of receive and transmit objects if unused objects are available.

Table 4-3 Classic mailbox layout

Note
n_{max} is the maximum amount of available mailboxes per CAN Controller and depends on the used derivative.

The “CanObjectId” (ECUc parameter) numbering is done in following order: FullCAN Tx, BasicCAN Tx, Unused, FullCAN Rx, BasicCAN Rx (like shown above). “CanObjectId’s” for next controller begin at end of last controller. Gaps in “CanObjectId” for unused mailboxes may occur.

4.3.2 Mailbox Processing Order

The hardware mailbox will be processed in following order:

Object Type	Order / priority to send or receive
Tx FullCAN	CAN ID Low to High
Tx BasicCAN	CAN ID Low to High
Rx FullCAN	Object ID Low to High
Rx BasicCAN	RxFifo Layout: FIFO Classic mailbox layout: Object ID Low to High

Note
Rx and Tx Polling Mode:

FullCANs will be processed before BasicCANs.

Tx Interrupt Mode:

BasicCANs will be processed before FullCANs.

Rx Interrupt Mode:

RxFifo: BasicCANs will be processed before FullCANs.

Classic mailbox layout: FullCANs will be processed before BasicCANs.

The order between Rx and Tx mailboxes depends on the call order of the polling tasks or the interrupt context and cannot be guaranteed.

The Rx Queue will work like a FIFO filled with the above mentioned method.

4.3.3 Acceptance Filter for BasicCAN

4.3.3.1 RxFifo Mailbox Layout

The Rx BasicCAN object is represented by the RxFifo of the CAN controller. Therefore from 8 up to 128 acceptance filters can be used to decide which messages are received by the CAN controller. Depending on the amount of used filters the RxFifo occupies a certain amount of mailboxes from the top of the mailbox layout. The amount of filters is independent

from the used message ID mode. Each filter can be used either for Standard or for Extended IDs.

Note
MICROSAR403 only: A maximum of 32 full configurable acceptance filters can be added to the Rx BasicCAN object.

See chapter section 4.9.2 and 7.4.5.1 or hardware manual for further information. Use feature “Rx BasicCAN Support” to deactivate unused code (for optimization).

4.3.3.2 Classic Mailbox Layout

Each Rx BasicCAN object has its individual reception mask. As a consequence for every Rx BasicCAN object an acceptance filter can be configured. For standard or extended ID systems there is one Rx BasicCAN objects available, two for mixed ID systems and multiple ones if the feature “Multiple BasicCAN” is enabled.

Note
MICROSAR403 only: Please ensure that only one filter is referenced to each Rx BasicCAN object. Especially when the feature CAN-FD was active and is disabled in the configuration tool.

If no message should be received, select the “Multiple Basic CAN” feature and set the amount to 0. Otherwise the filter should be set to “close”. Use the feature “Rx BasicCAN Support” to deactivate unused code (for optimization).

4.3.4 Remote Frames

Remote Frames rejected in Hardware	Remote Frames rejected in Software
X	

The CAN driver initializes the CAN controller to NOT receive remote frames. Therefore no additional action is required during runtime by the CAN driver for remote frame filtering. Remote frames will not have any influence on communication.

4.4 States / Modes

You can change the CAN cell mode via Can_SetControllerMode(). The last requested transition will be executed. The Upper layer has to take care about valid transitions.

Following modes changes are supported:

```
CAN_T_START
CAN_T_STOP
```

MICROSAR4 only: Notification of mode change may occur asynchronous by notification `CanIf_ControllerModeIndication()`

4.4.1 Start Mode (Normal Running Mode)

This is the mode where communication is possible. This mode has to be set after Initialization because Controller is first in stop-mode. See section 4.8.3 for hardware loops that may occur during start mode transition.

4.4.2 Stop Mode

If stop mode is requested then the CAN module is switched into Freeze mode. The CAN Controller is waiting for ongoing reception and transmission processes to be finished until the Freeze mode is entered successfully. This mode transition is monitored by a particular Hardware Loop Check. See section 4.8.3 for hardware loops that may occur during stop mode transition. In stop mode no transmission or reception is possible and incoming message frames are not acknowledged. All pending transmissions, receive and status indications are cancelled.

4.4.3 Sleep Mode

Wakeup by bus feature is not supported due to CAN controller characteristics, so the controller went into stop mode instead of sleep.

4.4.4 Bus Off

`CanIf_ControllerBusOff()` is called when the controller detects a Bus Off event. The mode is automatically changed to stop mode. The upper layers have to care about returning to normal running mode by calling start mode.

Please note:
Autosar 3 and 401: After switching to start mode the CAN module will be ready to transmit frames after it has detected 11 consecutive recessive bits on the bus. The intention is to be ready for transmission as soon as possible.
Autosar 403: After a Bus Off event occurs, the CAN Controller is waiting for 128x11 consecutive recessive bits on the bus for recovery (ISO compliant recovery time). The Bus Off recovery is performed asynchronously in start mode transition.

4.4.5 Silent Mode

Support API (`Can_SetSilentMode()`) to switch into ‘SilentMode’ where the controller does not take part on BUS communication (no ACK) but can listen for messages.
Refer to ISO 11898 bus monitoring.
Change in ‘SilentMode’ and back again to normal operating mode is only possible in Stop Mode. To get RX notifications in ‘SilentMode’ it is necessary to switch to Start Mode.
In ‘SilentMode’ it is possible to change the baud rate and switch to START mode, used to scan baud rates by detect reception.

Caution
Don't use any other API than `Can_SetSilentMode(CAN_SILENT_INACTIVE)`, `Can_SetControllerMode(START or STOP)`, `Can_ChangeBaudrate()` or `Can_SetBaudrate()` while silent mode is active – especially do not request any transmission.

The FlexCAN describes this mode as Listen-Only Mode:

After Entering Listen-Only Mode the controller operates in Error Passive mode. The error counters are frozen and will not be updated. So, the error counters cannot be used to detect a wrong baudrate.

In Listen-Only Mode the controller does not take part in the bus communication. In this mode transmission is disabled and the controller is recessive on the bus.

Only messages acknowledged by another node in the network will be received. There is no self ACK or internal rerouting. This means there must be nodes in the network that are able to communicate with each other with the correct baudrate set. Receiving such a message indicates that the correct baudrate is found.

Limitations
Only messages acknowledged by another node in the network will be received (there are already nodes in the network that are able to communicate with each other with the correct baudrate set).

Error counters cannot be used to detect a wrong baudrate.

4.5 Re-Initialization

A call to `Can_InitController()` cause a re-initialization of a dedicated CAN controller. Pending messages may be processed before the transition will be finished. A re-initialization is only possible out of Stop Mode and does not change to another Mode.

After re-initialization all CAN communication relevant registers are set to initial conditions.

4.6 CAN Interrupt Locking

`Can_DisableControllerInterrupts()` and `Can_EnableControllerInterrupts()` are used to disable and enable the controller specific Interrupt, Rx, Tx, Wakeup and Bus Off (/ Status) together. These functions can be called nested.

4.7 Main Functions

`Can_MainFunction_Write()`, `Can_MainFunction_Read()`, `Can_MainFunction_BusOff()` and `Can_MainFunction_Wakeup()` called by upper

layer to poll the events if the specific polling mode is activated. Otherwise these functions return without action and the events will be handled in interrupt context.

When individual polling is activated only mailboxes that are configured as to be polled will be polled in the main functions "`Can_MainFunction_Write()`" and "`Can_MainFunction_Read()`" all others handled in interrupt context.

When Rx Queue is activated the queue is filled in interrupt or polling context like configured. But The processing (indications) will be done in "`Can_MainFunction_Read()`" context.

MICROSAR4 only: Can_MainFunction_Mode() called by upper layer to poll asynchronous mode transition notifications.

4.8 Error Handling

4.8.1 Development Error Reporting

Development errors are reported to DET using the service `Det_ReportError()`, if the pre-compile parameter CAN_DEV_ERROR_DETECT == STD_ON.

The tables below, shows the API ID and Error ID given as parameter for calling the DET.

Instance ID is always 0 because no multiple Instances are supported.

Errors reported to DET:	
Error ID	**Short Description**
CAN_E_PARAM_POINTER	API gets an illegal pointer as parameter.
CAN_E_PARAM_HANDLE	API get an illegal handle as parameter
CAN_E_PARAM_DLC	API get an illegal DLC as parameter
CAN_E_PARAM_CONTROLLER	API get an illegal controller as parameter
CAN_E_UNINIT	Driver API is used but not initialized
CAN_E_TRANSITION	Transition for mode change is illegal
CAN_E_DATALOST (value: 0x07, AutoSar extension)	Rx overrun (overwrite) detected
CAN_E_PARAM_BAUDRATE (value: 0x08, AutoSar extension)	Selected Baudrate is not valid
CAN_E_RXQUEUE (value: 0x10, AutoSar extension)	Rx Queue overrun (last received message is lost, and will not be received → increase queue size)
CAN_E_TIMEOUT_DET (value: 0x11, AutoSar extension)	Same as CAN_E_TIMEOUT for DEM but this is notified to DET due to switch "CAN_DEV_TIMEOUT_DETECT" is set to STD_ON (see configuration options)

Table 4-4 Errors reported to DET

API from which the errors reported to DET:	
API ID	**Functions using that ID**
CAN_VERSION_ID	Can_GetVersionInfo()

CAN_INIT_ID	Can_Init()
CAN_INITCTR_ID	Can_InitController()
CAN_SETCTR_ID	Can_SetControllerMode()
CAN_DIINT_ID	Can_DisableControllerInterrupts()
CAN_ENINT_ID	Can_EnableControllerInterrupts()
CAN_WRITE_ID	Can_Write(), Can_CancelTx()
CAN_TXCNF_ID	CanHL_TxConfirmation()
CAN_RXINDI_ID	CanBasicCanMsgReceived(), CanFullCanMsgReceived()
CAN_CTRBUSOFF_ID	CanHL_ErrorHandling()
CAN_CKWAKEUP_ID	CanHL_WakeUpHandling(), Can_Cbk_CheckWakeup()
CAN_MAINFCT_WRITE_ID	Can_MainFunction_Write()
CAN_MAINFCT_READ_ID	Can_MainFunction_Read()
CAN_MAINFCT_BO_ID	Can_MainFunction_BusOff()
CAN_MAINFCT_WU_ID	Can_MainFunction_Wakeup()
CAN_MAINFCT_MODE_ID	Can_MainFunction_Mode()
CAN_CHANGE_BR_ID	Can_ChangeBaudrate()
CAN_CHECK_BR_ID	Can_CheckBaudrate()
CAN_SET_BR_ID	Can_SetBaudrate()
CAN_HW_ACCESS_ID (value: 0x20, AUTOSAR extension)	Used when hardware is accessed (call context is unknown)

Table 4-5 API from which the Errors are reported

4.8.1.1 Parameter Checking

AUTOSAR requires that API functions check the validity of their parameters (Refer to [1]). These checks are for development error reporting and can be enabled and disabled separately. Refer to the configuration chapter where the enabling/disabling of the checks is described. Enabling/disabling of single checks is an addition to the AUTOSAR standard which requires enable/disable the complete parameter checking via the parameter `CAN_DEV_ERROR_DETECT`.

4.8.1.2 Overrun/Overwrite Notification

As AUTOSAR extension the overrun detection may be activated by configuration tool. The notification can be configured to call an DET (MICROSAR4x) or Application call `Appl_CanOverrun`() or `Appl_CanFullCanOverrun`().

Please note
For BasicCAN reception:
RxFifo Mailbox Layout: If an overrun occurs the older message is received.
Classic Mailbox Layout: If an overrun occurs the newer message is received.
For FullCAN reception:
If an overrun occurs the newer message is received.

Due to the characteristics of the CAN Controller it can appear that in particular configurations FullCAN overruns are not detected by the CAN Controller and so cannot be indicated to upper layers. To ensure that every FullCAN overrun is detected the acceptance filter settings must be configured that none of the FullCAN messages could pass them. If not then the FullCAN message that should cause an overrun is received by the Rx BasicCAN mailbox and is ignored by the CAN driver. In this case the older message is received without overrun notification.

4.8.2 Production Code Error Reporting

Production code related errors are reported to DEM using the service `Dem_ReportErrorStatus()`, if the pre-compile parameter `CAN_PROD_ERROR_DETECT == STD_ON`.

The table below shows the Event ID and Event Status given as parameter for calling the DEM. This callout may occur in the context of different API calls (see Chapter “Hardware Loop Check”).

Event ID	Event Status	Short Description
CAN_E_TIMEOUT	DEM_EVENT_STATUS_FAILED	Timeout in “Hardware Loop Check” occurred, hardware has to be checked or timeout is too short.

Table 4-6 Errors reported to DEM

4.8.3 Hardware Loop Check / Timeout Monitoring

The feature “Hardware Loop Check” is used to break endless loops caused by hardware issue. This feature is configurable see Chapter 7 and also Timeout Duration description.

Since AUTOSAR4, synchronous part of mode transitions will be also limited by this timeout mechanism, which is no issue but a timing limit. (following asynchronous part of mode transition is handled without HardwareLoopCheck)

The Hardware Loop Check will be handled by CAN driver internal except when setting “Hardware Loop Check by Application” is activated.

Nevertheless refer to “short description” below there may be activities that should be initiated by the application (e.g. reset of CAN controller or special mode transitions). In this case the

“Hardware Loop Check by Application” is recommended to be used to handle the explicit loop like described.

4.8.3.1 Critical Loops

A loop exception has to be handled by application like described below.

Loop Name / source	Short Description
kCanLoopInit	This loop is used in function Can_InitController() when the CAN driver requests the soft reset of the CAN controller. - After soft reset is requested the CAN controller resets its internal state machines and resets the following registers: CANx_MCR (except the MDIS bit), CANx_ESR, CANx_IMRL, CANx_IMRH, CANx_IFRL and CANx_IFRH. - Configuration registers that control the interface to the CAN bus are not affected by soft reset. The following registers are unaffected: CANx_CTRL, CANx_RXGMASK, CANx_RX14MASK, CANx_RX15MASK and all message buffers. - Soft reset is synchronous and has to follow a request/acknowledge procedure across clock domains, So it may take some time to fully propagate its effect. - The SOFTRST bit remains asserted while reset is pending, and is automatically negated when reset completes. - In case of failure the application needs to restart with power on initialization.
kCanLoopEnterFreezeModeInit	This loop is used to monitor the mode transition to Freeze mode. - Call context: Can_InitController () - Expected duration: one CAN frame including interframe space. - Hardware condition for mode transition: “Waits to be in either intermission, passive error, bus off or idle state” (citation from hardware manual of semiconductor manufacturer). Consequential a dominant level at the Rx pin caused by the transceiver or other external influences may lead to longer blocking periods. - In case of failure the application needs to restart with power on initialization.
kCanLoopEnterDisableMode	This loop is used to monitor the mode transition to Disable mode. - Call context: Can_InitController () - Expected duration: some internal clock cycles - In case of failure the application needs to restart with power on initialization.

Loop Name / source	Short Description
kCanLoopLeaveDisableMode	This loop is used to monitor the mode transition to leave Disable mode. - Call context: Can_InitController () - Expected duration: some internal clock cycles - In case of failure the application needs to restart with power on initialization.

4.8.3.2 Uncritical Loops

No additional application handling needed after loop break.

Loop Name / source	Short Description
kCanLoopMsgReception	It is used to control message reception: - After a message frame has been received into the serial message buffer SMB the CAN hardware transfers its data from the SMB to the matching receive buffer (BasicCAN or FullCAN). During this transfer the CAN message buffer is set busy. The CAN driver must wait until the message buffer is no longer busy to process the message buffer content. - Loop index is used in the functions CanFullCanMsgReceived() and (if Classic Mailbox Layout is used) CanBasicCanMsgReceived() to wait until the message is completely received in the message buffer.

4.8.3.3 Loops used for synchronous mode transitions

AUTOSAR4: Driver handle the mode transition in asynchronous way afterwards, so no additional handling is necessary.

AUTOSAR3: Higher layer has to handle this (see below).

Loop Name / source	Short Description
kCanLoopStart	MICROSAR3: - Used while transition in mode ‘START’. - Call context: Can_SetControllerMode() - Expected duration: 11 recessive bits on the bus - Action after timeout: repeat state change MICROSAR4: - Used for short time mode transition blocking (short synchronous timeout). - Call context: Can_SetControllerMode() - Same value for kCanLoopStart and kCanLoopStop. - No Issue when timeout occur.
kCanLoopStop	MICROSAR3: - Used while transition in mode ‘STOP’.

Loop Name / source	Short Description
	- Call context: Can_SetControllerMode() - Expected duration: one CAN frame including interframe space. - Hardware condition for mode transition: “Waits to be in either intermission, passive error, bus off or idle state” (citation from hardware manual of semiconductor manufacturer). Consequential a dominant level at the Rx pin caused by the transceiver or other external influences may lead to longer blocking periods. MICROSAR4: - Used while transition in mode ‘STOP’. - Call context: Can_SetControllerMode() - Used for short time mode transition blocking (short synchronous timeout). - Same value for kCanLoopStart and kCanLoopStop. - No Issue when timeout occur.

Table 4-7 Hardware Loop Check

4.8.3.4 Calculation of Timeout Loops (Microsar3 only)

The loop duration must be at least maximum time of CAN message frame transmission.

T_{Bus} = length of frame / baudrate

Time for one loop cycle:

T_{Loop} = number instructions * number of average instruction CPU cycles / CPU frequency

Number of loops which will be executed while waiting for the break condition:

Number of Loops: $N_{Loop} = \frac{T_{Bus}}{T_{Loop}}$

Example

CPU clock = 100 MHz = 100000000 $\frac{instructions}{s}$

CAN baudrate = 100kBaud = 100 $\frac{kbit}{s}$

Assumption: 32 instructions for a single loop (varies for different optimization levels)

For a worst case estimation, we assume a message length of 150 bits.

CAN message frame transmission duration: $T_{Bus} = \frac{150bit}{100\frac{kbit}{s}} = 1.5ms$

Duration of a single loop: $T_{Loop} = \frac{32instructions}{100000000\frac{instructions}{s}} = 32\mu s$

Number of loops: loop: $N_{Loop} = \frac{1.5ms}{32\mu s} = 4688$

A much higher value than 4688 for the timeout duration is recommended in order not to break the loop in case of no error.

4.8.4 CAN RAM Check

The CAN driver supports a check of the CAN controller's mailboxes. The CAN controller RAM check is called internally every time a power on is executed within function Can_InitController(), or a Bus-Wakeup event happen. The CAN driver verifies that no used mailboxes are corrupt. A mailbox is considered corrupt if a predefined pattern is written to the appropriate mailbox registers and the read operation does not return the expected pattern. If a corrupt mailbox is found the function Appl_CanCorruptMailbox() is called. This function tells the application which mailbox is corrupt.

After the check of all mailboxes the CAN driver calls the call back function Appl_CanRamCheckFailed() if at least one corrupt mailbox was found. The application must decide if the CAN driver disables communication or not by means of the call back function's return value. If the application has decided to disable the communication there is no possibility to enable the communication again until the next call to Can_Init() or a wakeup event occurs.

The CAN RAM check functionality itself can be activated via Generation Tool.

4.8.5 Extended RAM Check

The CAN driver supports a check for all accessible CAN controller's control registers and mailbox registers. The extended RAM check will be executed while power on initialization and by direct call. Mailboxes will be deactivated when pattern check or configured values are corrupt. The CAN controller will be deactivated when one mailbox is corrupt or the controller's registers failed the pattern check or configured values are corrupt. Mailbox and controller stay deactivated until they are explicit activated again.

API to execute extended RAM Check:

`Can_RamCheckExecute()`

Callouts to notify corrupt mailboxes or controllers:

`CanIf_RamCheckCorruptController(), CanIf _RamCheckCorruptMailbox()`

API to activate the mailbox or controller again:

`Can_RamCheckEnableMailbox(),Can_RamCheckEnableController()`

(See also API description)

The CAN RAM check functionality itself can be activated via Generation Tool.

4.9 Hardware Specific

4.9.1 Tx Arbitration Start Delay (TASD)

The first bit of the CRC field on the CAN bus is one of the trigger events for the CAN controller to initiate an arbitration cycle. For this case the arbitration can be delayed to optimize the performance of transmission proceedings. If an arbitration process is delayed there is a little more time left for Tx message buffer configuration but less time is reserved for the arbitration procedure.

Please note
The default TASD value is 16. The default value for TASD is used if the TASD computation in the generation tool stays disabled, see chapter 7.4.2.

4.9.2 RxFifo

The CAN driver supports the RxFifo feature which is available in the CAN controller. The RxFifo is used as Rx BasicCAN object if the RxFifo Mailbox Layout is used. A storage capacity of 6 message frames is provided including message frame receive order handling. It follows that the application has more time to process message frame reception without message lost. The RxFifo has a powerful filter usage. Up to 128 filters can be configured to handle the message reception.

Please note
Not every filter of the RxFifo is full configurable. Due to hardware characteristics some filters cannot be assembled by an own (individual) mask. These filters share a dedicated global mask that is configured to “must match”. They cannot be configured by hand in the generation tool but are only used by the acceptance filter optimization process. More details see hardware manual or section 7.4.4 and 7.4.5.1.

4.9.3 Error Interrupt

The FlexCAN error interrupt source is not used by the CAN driver. Only BusOff events are handled and reported to the upper layers by the CAN driver.

4.9.4 Privileged Operating Modes

The CPU supports different operating modes (e.g. User and Supervisor mode). Depending on the derivative the R/W access to some CAN hardware registers are possibly allowed in the Supervisor mode only. Depending on the system design the CAN driver may not be allowed to switch into a privileged mode by itself. The protected registers must be accessed outside the CAN driver from an upper layer (for example handled by the Operating System) which is privileged to switch in the Supervisor mode.

4.9.4.1 Privileged Access with OS Callbacks

The layer (for example the Operating System) which is responsible to handle the privileged access to the protected registers must define the following area defines depending on the used hardware channels:

Hardware Controller	Area Defines
FlexCAN_0	CAN_PROTECTED_AREA_CH0
FlexCAN_1	CAN_PROTECTED_AREA_CH1
FlexCAN_2	CAN_PROTECTED_AREA_CH2
FlexCAN_3	CAN_PROTECTED_AREA_CH3
FlexCAN_4	CAN_PROTECTED_AREA_CH4
FlexCAN_5	CAN_PROTECTED_AREA_CH5
FlexCAN_6	CAN_PROTECTED_AREA_CH6
FlexCAN_7	CAN_PROTECTED_AREA_CH7

Table 4-8 Hardware Controller – Area Defines

For each hardware channel the following registers must be located inside the privileged region that can be configured in the Operating System module of the generation tool:

FlexCAN register	Physical address
CAN_MCR	Base address + 0x0000
CAN_CTRL1	Base address + 0x0004
CAN_ESR1	Base address + 0x0020

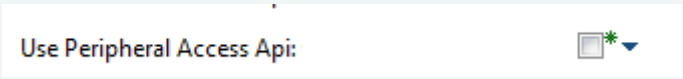
Table 4-9 Protected CAN registers

The OS callbacks that are called by the CAN driver are provided by the Operating System component. Please refer to Technical Reference of the OS component to get further information.

Note

Microsar403:

For Microsar403 the feature can be configured in the configuration tool with the boolean attribute “Use Peripheral Access Api”.



Microsar401 and Microsar3:

To enable the CAN driver to operate in User mode the following code segment must be added to user configuration file:

```
#include "Os.h" (or the corresponding layer that serves the area defines)
#define C_ENABLE_USER_MODE_OS

#define CAN_PROTECTED_MODE STD_ON

extern CONST(uint16, CAN_CONST) CanProtectedAreaId[]; /*
PRQA S 3684 */ /* empty brackets needed to avoid compile
optimization because of library build */
```

4.9.4.2 Privileged Access with Application Callbacks

The access to protected registers can also be performed with application callbacks. In this case the application is responsible to handle the privileged access to the protected registers. The callback functions are described in the sections 6.2.42, 6.2.43, 6.2.44 and 6.2.45.

Note

To enable the CAN driver to operate in User mode the following code segment must be added to user configuration file:

```
#define C_ENABLE_USER_MODE_APPL
```

4.9.4.3 Peripheral Bridge

For use of the driver in a Microsar Safe context (core runs in user mode) for some derivatives it is necessary to configure the Peripheral Bridge to have access to the FlexCAN controller. Configuration of the Peripheral Bridge is not part of the driver and must be done by the OS. Please refer to the reference manual of the used derivative.

4.10 Virtual Addressing

Virtual Addressing is intended to be used in combination with operating systems that use virtual addresses instead of physical addresses to access the FlexCAN3 hardware. The driver sets the virtual base address by calling the application callback function ApplCanPowerOnGetBaseAddress() during Can_Init(). This function needs to be declared and defined by the application. It is described in section 6.2.46.

Note

To enable the Virtual Addressing feature the following code segment must be added to user configuration file:

```
#define C_ENABLE_UPDATE_BASE_ADDRESS
```

5 Integration

This chapter gives necessary information for the integration of the MICROSAR CAN into an application environment of an ECU.

5.1 Scope of Delivery

The delivery of the CAN contains the files, which are described in the chapter's 5.1.1 and 5.1.2:

Dependent on library or source code delivery the marked (+) files may not be delivered.

5.1.1 Static Files

File Name	Description
(+) Can_Local.h	This is an internal header file which should not be included outside this module
(+) Can.c	This is the source file of the CAN. It contains the implementation of CAN module functionality.
(+) Can.lib	This is the library build out of Can.c, Can.h and Can_Local.h
Can.h	This is the header file of the CAN module (include API declaration)
Can_Hooks.h	This is the header file to define the Hook-functions or macros. (this is a project specific file and may not exist)
Can_Irq.c	This is the interrupt declaration and callout file (supports interrupt configuration as link time settings)

Table 5-1 Static files

5.1.2 Dynamic Files

The dynamic files are generated by the configuration tool [GENy].

File Name	Description
Can_Cfg.h	Generated header file, contains some type, prototype and pre-compile settings
Can_Lcfg.c	Generated file contains link time settings.
Can_PBcfg.c	Generated file contains post build settings.
Can_DrvGeneralTypes.h	Generated file contains CAN driver part of Can_GeneralTypes.h (supported by Integrator)

Table 5-2 Generated files

5.2 Include Structure

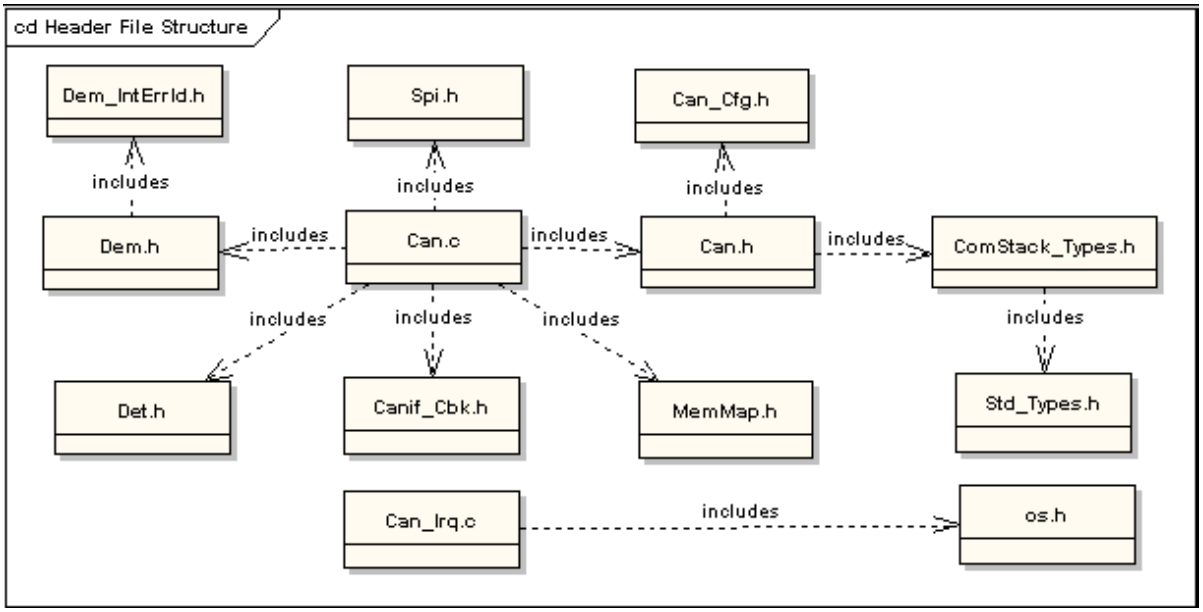


Figure 5-1 Include Structure (AUTOSAR)

Deviation from AUTOSAR specification:

- Additionally the EcuM_Cbk.h is included by Can_Cfg.h (needed for wakeup notification API).
- ComStack_Types.h included by Can_Cfg.h, because the specified types have to be known in generated data as well.
- MICROSAR4x only: Os.h will be included by Can_Cfg.h because of used data-types
- Spi.h is not yet used.
- Additionally the file Can_Hooks.h may be included by Can.c.
- MICROSAR403 only: Can_GeneralTypes.h will be included by Can_Cfg.h not by Can.h direct.

5.3 Critical Sections

The AUTOSAR standard provides with the BSW Scheduler a BSW module, which handles entering and leaving critical sections.

For more information about the BSW Scheduler please refer to [3]. When the BSW Scheduler is used the CAN Driver provides critical section codes that have to be mapped by the BSW Scheduler to following mechanism:

Critical Section Define	Description
CAN_EXCLUSIVE_AREA_0	Using inside message reception context in the functions CanFullCanMsgReceived() and (if Classic Mailbox Layout is used) CanBasicCanMsgReceived(). > Duration is short (< 20 instructions). > No API call of other BSW inside. > Disable all interrupts that serves CAN functionality.
CAN_EXCLUSIVE_AREA_1	Using inside Can_DisableControllerInterrupts() and Can_EnableControllerInterrupts() to secure Interrupt counters for nested calls. > Duration is short (< 20 instructions). > No API call of other BSW inside. > Disable global interrupts – or – Empty in case Can_Disable/EnableControllerInterrupts() are called within context with lower or equal priority than CAN interrupt.
CAN_EXCLUSIVE_AREA_2	Using inside Can_Write() to secure software states of transmit objects. > Only when no Vector CAN Interface is used. > Duration is medium (< 50 instructions). > No API call of other BSW inside. > Disable global interrupts - or - Disable CAN interrupts and does not call function reentrant.
CAN_EXCLUSIVE_AREA_3	Using inside Tx confirmation to secure state of transmit object in case of cancellation. (Only used when Vector Interface Version smaller 4.10 used) > Duration is medium (< 50 instructions). > Call to CanIf_CancelTxConfirmation() inside (no more calls in CanIf). > Disable global interrupts - or - Disable CAN interrupts and do not call function Can_Write() within.
CAN_EXCLUSIVE_AREA_4	Using inside received data handling (Rx Queue treatment) to secure Rx Queue counter and data. > Duration is short (< 20 instructions). > No API call of other BSW inside. > Disable Global Interrupts - or - Disable all CAN interrupts.

CAN_EXCLUSIVE_AREA_5	Using inside wakeup handling to secure state transition. (Only in wakeup polling mode) > Duration is short (< 10 instructions). > Call to DET inside. > Disable global interrupts (do not use CAN interrupt locks here)
CAN_EXCLUSIVE_AREA_6	Using inside Can_SetControllerMode() and BusOff to secure state transition. > Duration is medium (< 100 instructions). > No API call of other BSW inside. > Use CAN interrupt locks here, when the API for one controller is not called in a context higher than the CAN interrupt or Disable global interrupts

Table 5-3 Critical Section Codes

5.4 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions defined for the CAN Interface and illustrates their assignment among each other.

Compiler Abstraction Definitions / Memory Mapping Sections	CAN_CODE	CAN_STATIC_CODE	CAN_CONST	CAN_CONST_PBCFG	CAN_VAR_NOINIT	CAN_VAR_INIT	CAN_INT_CTRL	CAN_REG_CANCELL	CAN_RX_TX_DATA	CAN_APPL_CODE	CAN_APPL_CONST	CAN_APPL_VAR
CAN_START_SEC_CODE CAN_STOP_SEC_CODE	■											
CAN_START_SEC_STATIC_CODE CAN_STOP_SEC_STATIC_CODE		■										
CAN_START_SEC_CONST_8BIT CAN_STOP_SEC_CONST_8BIT			■									
CAN_START_SEC_CONST_16BIT CAN_STOP_SEC_CONST_16BIT			■									
CAN_START_SEC_CONST_32BIT CAN_STOP_SEC_CONST_32BIT			■									
CAN_START_SEC_CONST_UNSPECIFIED CAN_STOP_SEC_CONST_UNSPECIFIED			■									
CAN_START_SEC_PBCFG				■								

CAN_STOP_SEC_PBCFG												
CAN_START_SEC_PBCFG_ROOT CAN_STOP_SEC_PBCFG_ROOT				■								
CAN_START_SEC_VAR_NOINIT_UNSPECIFIED CAN_STOP_SEC_VAR_NOINIT_UNSPECIFIED					■							
CAN_START_SEC_VAR_INIT_UNSPECIFIED CAN_STOP_SEC_VAR_INIT_UNSPECIFIED						■						
CAN_START_SEC_CODE_APPL CAN_STOP_SEC_CODE_APPL										■		

Table 5-4 Compiler abstraction and memory mapping

The Compiler Abstraction Definitions CAN_ APPL_CODE, CAN_ APPL_VAR and CAN_ APPL_CONST are used to address code, variables and constants which are declared by other modules and used by the CAN driver.

These definitions are not mapped by the CAN driver but by the memory mapping realized in the CAN Interface or direct by application.

CAN_ CODE: used for CAN module code.

CAN_ STATIC_CODE: used for CAN module local code.

CAN_ CONST: used for CAN module constants.

CAN_ CONST_PBCFG: used for CAN module constants in Post-Build section.

CAN_ VAR_*: used for CAN module variables.

CAN_ INT_CTRL: is used to access the CAN interrupt controls.

CAN_ REG_CANCELL: is used to access the CAN cell itself.

CAN_ RX_TX_DATA: access to CAN Data buffers.

CAN_ APPL_*: access to higher layers.

5.5 Hardware Specific Hints

For the integration of the CAN interrupt service functions please refer to section 6.1.5.

The CAN pin configuration settings vary from derivative to derivative. Please refer to the hardware manual of the semiconductor manufacturer to get detailed information.

6 API Description

6.1 Interrupt Service Routines provided by CAN

Depend on the settings in Tools component e.g. Hw_ImxCpu, the interrupt routine is given by the driver or by Operating System. (Selection below, not MICROSAR403)

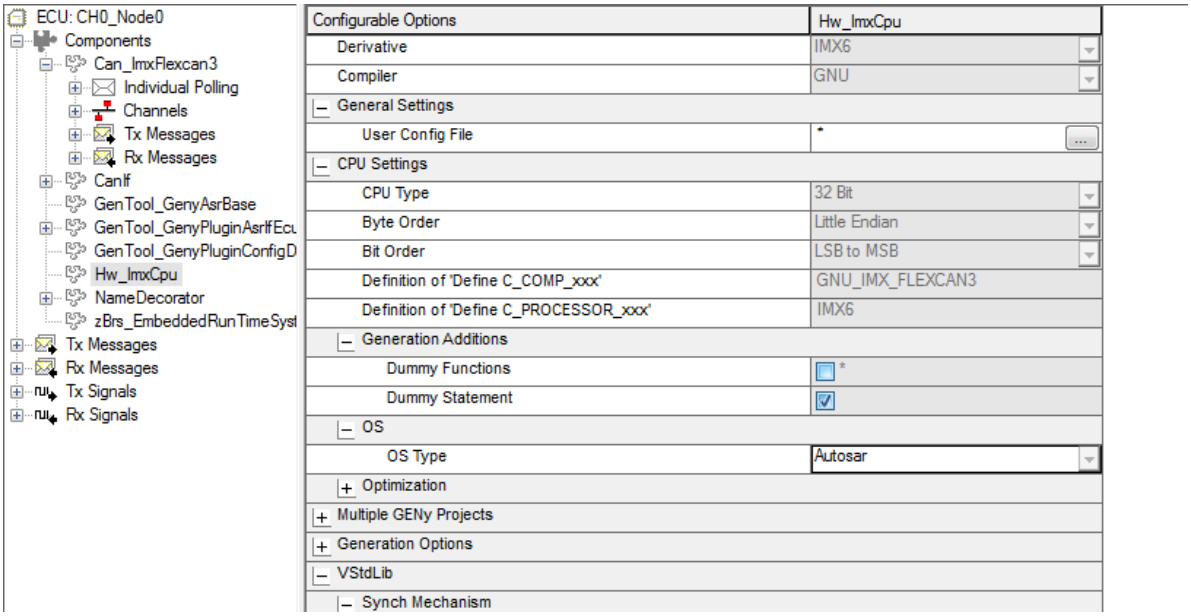


Figure 6-1 Select OS Type

There is the possibility to choose OS Type. Please select “None” for using no OS, “Autosar” for AUTOSAR OS or “OSEK” for OSEK OS systems.

6.1.1 OSEK (OS)

This means to include osek.h.

Switch: V_OSTYPE_OSEK

6.1.2 AutoSar (OS)

Os.h header file is used.

Switch: V_OSTYPE_AUTOSAR

6.1.3 None (OS)

Choose “None” for OS Type, to include no Os header files and have no category 2 interrupt.
Switch: V_OSTYPE_NONE

6.1.4 Type of Interrupt Function

See Chapter “Component Settings” for configuration aspects.

- Category 2 (only for OSEK OS or AUTOSAR OS):
  A macro “ISR(Can<type>Isr_x)” will be used to declare ISR function call. The name given as parameter for interrupt naming (x = Physical CAN Channel number, type = interrupt type: BufOff, Mailbox). For macro definition see OS specification. The OS

has full control of the ISR.
switch: C_ENABLE_OSEK_OS_INTCAT2

> Category 1:
Using OS with category 1 interrupts need an Interface layer handling these interrupts in task context like defined in BSW00326 (AUTOSAR_SRS_General).
switch: C_DISABLE_OSEK_OS_INTCAT2

> Void-Void Interrupt Function:
Like in Category 1 the Interrupt is not handled by OS and the ISR is declared as void ISR(void) and has to be called by interrupt controller in case of an CAN interrupt.
switch: C_ENABLE_ISRVOID

6.1.5 Interrupt functions

The CAN driver does not provide ISR functions. The operating system or the application has to provide an interrupt handler which calls the CAN driver subroutines for message frame and status event handling. S32K derivatives use CanBusOffIsr-<x> and CanMailboxIsr_<x> while IMX Derivatives have a combined interrupt CanIsr<x> for mailbox, busoff and wakeup handling.

6.1.5.1 CanBusOffIsr_<x>

Prototype	
`void` **`CanBusOffIsr_<x>`** `( void )`	
Parameter	
---	---
Return code	
---	---
Functional Description	
Handles interrupts of hardware channel <x> for BusOff events.	
Particularities and Limitations	
■ This function is not ended with a “RTI” instruction.	

6.1.5.2 CanMailboxIsr_<x>

Prototype	
`void` **`CanMailboxIsr_<x>`** `( void )`	
Parameter	
---	---
Return code	
---	---

Functional Description
Handles interrupts of hardware channel <x> for Rx and Tx events.
Particularities and Limitations
■ This function is not ended with an “RTI” instruction.

Please note
Refer to the following table for the correct mapping of interrupt service functions:

FLEXCAN<x>_ESR[ERR_INT]	Unused
FLEXCAN<x>_ESR_BOFF	CanBusOffIsr_<x>
FLEXCAN<x>_BUF_*	CanMailboxIsr_<x>

* the mapping of the mailboxes (message buffers) to interrupt lines can vary and depends on the used derivative. All interrupt lines that serve mailbox interrupt notifications must be mapped to CanMailboxIsr_<x>.

6.1.5.3 CanIsr_<x>

Prototype	
`void CanIsr_<x> ( void )`	
Parameter	
---	---
Return code	
---	---
Functional Description	
Handles interrupts of hardware channel <x> for Rx, Tx, Busoff and Wakeup events.	
Particularities and Limitations	
■ This function is not ended with an “RTI” instruction.	

Please note
Driver does not support Wakeup events.

6.1.5.4 IMX8 QuadXPlus/DualXPlus Interrupt Handling

Imx8 QuadXPlus/DualXPlus derivatives do not have dedicated CAN interrupts. Instead the CAN interrupt events arrive with External Interrupt 4 among other interrupt events. The procedure is to define category 2 interrupts so that the use of CanIsr<x> interrupts is enabled and then add the interrupts (Os_Isr_CanIsr_<x>) to the External Interrupt 4. A general instruction on how to do it cannot be given. For example, for the MCAL external interrupt 4 it is possible to redefine the CAN interrupt service routines and add the necessary flags in a user config file.

6.2 Services provided by CAN

6.2.1 Can_InitMemory

Prototype	
`void Can_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Power-up memory initialization.	
Particularities and Limitations	
Initializes component variables in *_INIT_* sections at power up. Use this to re-run the system without performing a new start from power on. (E.g.: used to support an ongoing debug session without a complete re-initialization.)	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-1 Can_InitMemory

6.2.2 Can_Init

Prototype	
`void Can_Init (Can_ConfigPtrType ConfigPtr)`	
Parameter	
ConfigPtr [in]	Component configuration structure
Return code	
void	none
Functional Description	
Initializes component.	
Particularities and Limitations	
> Interrupts are disabled.Module is uninitialized.Can_InitMemory() has been called unless CAN Module is initialized by start up code. Initializes all component variables and sets the component state to initialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-2 Can_Init

6.2.3 Can_InitController

Prototype	
`void Can_InitController (uint8 Controller, Can_ControllerBaudrateConfigPtrType Config)`	
Parameter	
Controller [in]	CAN controller
Config [in]	Pointer to baud rate configuration structure
Return code	
void	none
Functional Description	
Initialization of controller specific CAN hardware.	
Particularities and Limitations	
> Disabled Interrupts.Can_Init() has to be called. The CAN driver registers and variables are initialized. The CAN controller is fully initialized and left back within the state "STOP Mode", ready to change to "Running Mode". Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR40 Has to be called during the start up sequence before CAN communication takes place. Must not be called while in "SLEEP Mode". CREQ-969, CREQ-994	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-3 Can_InitController

6.2.4 Can_InitController

Prototype	
`void Can_InitController (uint8 Controller, Can_ControllerConfigPtrType ControllerConfigPtr)`	
Parameter	
Controller [in]	CAN controller
ControllerConfigPtr [in]	Pointer to baud rate configuration structure
Return code	
void	none
Functional Description	
Initialization of controller specific CAN hardware.	
Particularities and Limitations	
> Disabled Interrupts.Can_Init() has to be called. The CAN driver registers and variables are initialized. The CAN controller is fully initialized and left back within the state "STOP Mode", ready to change to "Running Mode". Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR30 Has to be called during the start up sequence before CAN communication takes place ". Must not be called while in "SLEEP Mode". CREQ-969, CREQ-994	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-4 Can_InitController

6.2.5 Can_ChangeBaudrate

Prototype	
`Std_ReturnType` **`Can_ChangeBaudrate`** `(uint8 Controller, const uint16 Baudrate)`	
Parameter	
Controller [in]	CAN controller to be changed
Baudrate [in]	Baud rate to be set
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not set
Std_ReturnType	E_OK Baud rate is set
Functional Description	
Change baud rate.	
Particularities and Limitations	
The CAN controller must be in "STOP Mode". This service shall change the baud rate and reinitialize the CAN controller. Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && ((CAN_CHANGE_BAUDRATE_API == STD_ON) || (CAN_SET_BAUDRATE_API == STD_OFF)) Has to be called during the start up sequence before CAN communication takes place but after calling "Can_Init()". CREQ-995	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-5 Can_ChangeBaudrate

6.2.6 Can_CheckBaudrate

Prototype	
`Std_ReturnType` **`Can_CheckBaudrate`** `(uint8 Controller, const uint16 Baudrate)`	
Parameter	
Controller [in]	CAN controller to be checked
Baudrate [in]	Baud rate to be checked
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not available
Std_ReturnType	E_OK Baud rate is available
Functional Description	
Checks baud rate.	
Particularities and Limitations	
The CAN controller must be initialized. This service shall check if the given baud rate is supported of the CAN controller. Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && (CAN_CHANGE_BAUDRATE_API == STD_ON) CREQ-995	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-6 Can_CheckBaudrate

6.2.7 Can_SetBaudrate

Prototype	
`Std_ReturnType` **`Can_SetBaudrate`** `(uint8 Controller, uint16 BaudRateConfigID)`	
Parameter	
Controller [in]	CAN controller to be set
BaudRateConfigID [in]	Identity of the configured baud rate (available as Symbolic Name)
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not set
Std_ReturnType	E_OK Baud rate is set
Functional Description	
Set baud rate.	
Particularities and Limitations	
The CAN controller must be in "STOP Mode". This service shall change the baud rate and reinitialize the CAN controller. (Similar to Can_ChangeBaudrate() but used when identical baud rates are used for different CAN FD settings). Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && (CAN_SET_BAUDRATE_API == STD_ON) CREQ-995	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-7 Can_SetBaudrate

6.2.8 Can_InitStruct

Prototype	
`void Can_InitStruct (uint8 Controller, uint8 Index)`	
Parameter	
Controller [in]	CAN controller to be changed
Index [in]	Index of the initialization structure to be used for baud rate and mask settings
Return code	
void	none
Functional Description	
Set active initialization structure.	
Particularities and Limitations	
Can_Init() was called. Set content of the initialization structure (before calling Can_InitController()). Service function to change the initialization structure set up left behind by the Generation Tool. The structure contains information about baud rate and filter settings. Subsequent Can_InitController() must be called to activate these settings. Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR30 Call this function between calling "Can_Init()" and "Can_InitController()". None AUTOSAR API CREQ-995	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-8 Can_InitStruct

6.2.9 Can_GetVersionInfo

Prototype	
`void` **`Can_GetVersionInfo`** `(Can_VersionInfoPtrType VersionInfo)`	
Parameter	
VersionInfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information.	
Particularities and Limitations	
-	
Returns version information (MICROSAR3: BCD coded), vendor ID and AUTOSAR module ID (MICROSAR3: instance ID) of the component.	
Configuration Variant(s): CAN_VERSION_INFO_API == STD_ON CREQ-991	
Call context	
> ANY	
> This function is Synchronous	
> This function is Reentrant	

Table 6-9 Can_GetVersionInfo

6.2.10 Can_GetStatus

Prototype	
`uint8 Can_GetStatus (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller requested for status information
Return code	
uint8	CAN_STATUS_START START mode
	CAN_STATUS_STOP STOP mode
	CAN_STATUS_INIT Initialized controller
	CAN_STATUS_INCONSISTENT STOP or SLEEP are inconsistent over common CAN controllers
	CAN_DEACTIVATE_CONTROLLER RAM check failed CAN controller is deactivated
	CAN_STATUS_WARNING WARNING state
	CAN_STATUS_PASSIVE PASSIVE state
	CAN_STATUS_BUSOFF BUSOFF mode
	CAN_STATUS_SLEEP SLEEP mode
Functional Description	
Get status/mode of the given controller.	
Particularities and Limitations	
-	
Delivers the status of the hardware, as a bit coded value where multiple bits may be set.	
Configuration Variant(s): CAN_GET_STATUS == STD_ON the return value can be analysed using the provided API macros:	
CAN_HW_IS_OK(), CAN_HW_IS_WARNING(), CAN_HW_IS_PASSIVE()	
CAN_HW_IS_BUSOFF(), CAN_HW_IS_WAKEUP(), CAN_HW_IS_SLEEP()	
CAN_HW_IS_STOP(), CAN_HW_IS_START(), CAN_HW_IS_INCONSISTENT()	
None AUTOSAR API CREQ-978	
Call context	
> ANY	
> This function is Synchronous	
> This function is Reentrant	

Table 6-10 Can_GetStatus

6.2.11 Can_SetControllerMode

Prototype	
`Can_ReturnType` **`Can_SetControllerMode`** `(uint8 Controller, Can_StateTransitionType Transition)`	
Parameter	
Controller [in]	CAN controller to be set
Transition [in]	Requested transition to destination mode CAN_T_START, CAN_T_STOP, CAN_T_SLEEP, CAN_T_WAKEUP
Return code	
Can_ReturnType	CAN_NOT_OK MICROSAR3: mode change unsuccessful - MICROSAR4: transition request rejected
Can_ReturnType	CAN_OK MICROSAR3: mode change successful - MICROSAR4: transition request accepted
Functional Description	
Change the controller mode.	
Particularities and Limitations	
Interrupts locked Request a mode transition witch processed synchronous for MICROSAR3 and return the success. Or do it asynchronous for MICROSAR4 and call a notification when successful. BUSOFF, WAKEUP and RAM check will be handled as well.	
Call context	
> ANY > This function is Non-Reentrant	

Table 6-11 Can_SetControllerMode

6.2.12 Can_ResetBusOffStart

Prototype	
`void Can_ResetBusOffStart (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller
Return code	
void	none
Functional Description	
Start BUSOFF handling.	
Particularities and Limitations	
- This is a compatibility function (for a CANbedded protocol stack) used during the start of the BUSOFF handling to remove the BUSOFF state. Configuration Variant(s): CAN_BUSOFF_SUPPORT_API == STD_ON Has to be called inside BUSOFF notification. Empty function when not configured. None AUTOSAR API CREQ-984	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-12 Can_ResetBusOffStart

6.2.13 Can_ResetBusOffEnd

Prototype	
`void Can_ResetBusOffEnd (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller
Return code	
void	none
Functional Description	
Finish BUSOFF handling.	
Particularities and Limitations	
Has to be called after Can_ResetBusOffStart(). This is a compatibility function (for a CANbedded protocol stack) used during the end of the BUSOFF handling to remove the BUSOFF state. Configuration Variant(s): CAN_BUSOFF_SUPPORT_API == STD_ON The call should be done 128*11 recessive CAN bit times later because the call may wait this time to handle BUSOFF. No nesting of Can_SetControllerMode() and Can_ResetBusOffEnd() is allowed. Empty function when not configured. None AUTOSAR API CREQ-984	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-13 Can_ResetBusOffEnd

6.2.14 Can_Write

Prototype	
`Can_ReturnType Can_Write (Can_HwHandleType Hth, Can_PduInfoPtrType PduInfo)`	
Parameter	
Hth [in]	Handle of the mailbox intended to send the message
PduInfo [in]	Information about the outgoing message (ID, dataLength, data)
Return code	
Can_ReturnType	CAN_NOT_OK transmit request rejected
	CAN_OK transmit request successful
	CAN_BUSY transmit request could not be accomplished due to the controller is busy.
Functional Description	
Send a CAN message.	
Particularities and Limitations	
disabled CAN interrupts / interrupts locked. copy data, DLC and ID to the send mailbox and request a transmission.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-14 Can_Write

6.2.15 Can_CancelTx

Prototype	
`void Can_CancelTx (Can_HwHandleType Hth, PduIdType PduId)`	
Parameter	
Hth [in]	Handle of the mailbox intended to be cancelled.
PduId [in]	PDU identifier
Return code	
void	none
Functional Description	
Cancel TX message.	
Particularities and Limitations	
- Cancel the TX message in the hardware buffer (if possible) or mark the message as not to be confirmed in case of the cancellation is unsuccessful.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-15 Can_CancelTx

6.2.16 Can_SetMirrorMode

Prototype	
`void Can_SetMirrorMode (uint8 Controller, CddMirror_MirrorModeType mirrorMode)`	
Parameter	
Controller [in]	CAN controller
mirrorMode [in]	Activate or deactivate the mirror mode.
Return code	
void	none
Functional Description	
Activate mirror mode.	
Particularities and Limitations	
-	
Switch the Appl_GenericPreCopy/Confirmation function ON or OFF.	
Configuration Variant(s): C_ENABLE_MIRROR_MODE (user configuration file) Called by "Mirror Mode" CDD.	
None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-16 Can_SetMirrorMode

6.2.17 Can_SetSilentMode

Prototype	
`Std_ReturnType` **`Can_SetSilentMode`** `(uint8 Controller, Can_SilentModeType silentMode)`	
Parameter	
Controller [in]	CAN controller
silentMode [in]	Activate or deactivate the silent mode with CAN_SILENT_ACTIVE, CAN_SILENT_INACTIVE (Enumeration)
Return code	
Std_ReturnType	E_OK mode change successful.
Std_ReturnType	E_NOT_OK mode change failed.
Functional Description	
Activate and deactivate the silent mode. Switch to silent mode, as a listen only mode without ACK, and deactivate this mode again for regular communication.	
Particularities and Limitations	
The CAN controller must be in stop mode. Configuration Variant(s): CAN_SILENT_MODE == STD_ON	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-17 Can_SetSilentMode

6.2.18 Can_CheckWakeup

Prototype	
`Std_ReturnType` **`Can_CheckWakeup`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
Std_ReturnType	E_OK the given controller caused a WAKEUP before.
Std_ReturnType	E_NOT_OK the given controller caused no WAKEUP before.
Functional Description	
Check WAKEUP occur.	
Particularities and Limitations	
- Check the occurrence of WAKEUP events for the given controller (used as WAKEUP callback for higher layers).	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-18 Can_CheckWakeup

6.2.19 Can_DisableControllerInterrupts

Prototype	
`void Can_DisableControllerInterrupts (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to disable interrupts for.
Return code	
void	none
Functional Description	
Disable CAN interrupts.	
Particularities and Limitations	
Must not be called while CAN controller is in SLEEP mode. Disable the CAN interrupt for the given controller (e.g. due to data consistency reasons).	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-19 Can_DisableControllerInterrupts

6.2.20 Can_EnableControllerInterrupts

Prototype	
`void Can_EnableControllerInterrupts (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to enable interrupts for.
Return code	
void	none
Functional Description	
Enable CAN interrupts.	
Particularities and Limitations	
Must not be called while CAN controller is in SLEEP mode. Re-enable the CAN interrupt for the given controller (e.g. due to data consistency reasons).	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-20 Can_EnableControllerInterrupts

6.2.21 Can_MainFunction_Write

Prototype	
`void Can_MainFunction_Write (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
TX message observation.	
Particularities and Limitations	
Must not interrupt the call of Can_Write(). Polling TX events (confirmation, cancellation) for all controllers and all TX mailboxes to accomplish the TX confirmation handling (like CanInterface notification).	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-21 Can_MainFunction_Write

6.2.22 Can_MainFunction_Read

Prototype	
`void Can_MainFunction_Read (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
RX message observation.	
Particularities and Limitations	
-	
Polling RX events for all controllers and all RX mailboxes to accomplish the	
RX indication handling (like CanInterface notification).	
Also used for a delayed read (from task level) of the RX Queue messages which were queued from interrupt context.	
Call context	
> TASK	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-22 Can_MainFunction_Read

6.2.23 Can_MainFunction_BusOff

Prototype	
`void Can_MainFunction_BusOff (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
BUSOFF observation.	
Particularities and Limitations	
- Polling of BUSOFF events to accomplish the BUSOFF handling. Service function to poll BUSOFF events for all controllers to accomplish the BUSOFF handling (like calling of CanIf_ControllerBusOff() in case of BUSOFF occurrence).	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-23 Can_MainFunction_BusOff

6.2.24 Can_MainFunction_Wakeup

Prototype	
`void Can_MainFunction_Wakeup (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
WAKEUP observation.	
Particularities and Limitations	
- Polling WAKEUP events for all controllers to accomplish the WAKEUP handling (like calling of "CanIf_SetWakeupEvent()" in case of WAKEUP occurrence).	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-24 Can_MainFunction_Wakeup

6.2.25 Can_MainFunction_Mode

Prototype	
`void Can_MainFunction_Mode (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Mode transition observation.	
Particularities and Limitations	
- Polling Mode changes over all controllers. (This is handled asynchronous if not accomplished in "Can_SetControllerMode()") Configuration Variant(s): MICROSAR4x only CREQ-978	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-25 Can_MainFunction_Mode

6.2.26 Can_RamCheckExecute

Prototype	
`void Can_RamCheckExecute (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be checked.
Return code	
void	none
Functional Description	
Start checking the CAN cells RAM.	
Particularities and Limitations	
Has to be called within STOP mode. Check all controller specific and mailbox specific registers by write patterns and read back. Issue notification will appear in this context.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-26 Can_RamCheckExecute

6.2.27 Can_RamCheckEnableMailbox

Prototype	
`void Can_RamCheckEnableMailbox (Can_HwHandleType htrh)`	
Parameter	
htrh [in]	CAN mailbox to be reactivated.
Return code	
void	none
Functional Description	
Reactivate a mailbox after RamCheck failed.	
Particularities and Limitations	
Has to be called within STOP mode after RamCheck failed (controller is deactivated). Mailbox will be reactivated by execute reinitialization.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-27 Can_RamCheckEnableMailbox

6.2.28 Can_RamCheckEnableController

Prototype	
`void Can_RamCheckEnableController (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be reactivated.
Return code	
void	none
Functional Description	
Reactivate CAN cells after RamCheck failed.	
Particularities and Limitations	
Has to be called within STOP mode after RamCheck failed (controller is deactivated). CAN cell will be reactivated by execute reinitialization.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-28 Can_RamCheckEnableController

6.2.29 Appl_GenericPrecopy

Prototype	
`Can_ReturnType Appl_GenericPrecopy (uint8 Controller, Can_IdType ID, uint8 DataLength, Can_DataPtrType DataPtr)`	
Parameter	
Controller [in]	CAN controller which received the message.
ID [in]	ID of the received message (include IDE,FD). In case of extended or mixed ID systems the highest bit (bit 31) is set to mark an extended ID. FD-bit (bit 30) can be masked out with user define CAN_ID_MASK_IN_GENERIC_CALLOUT.
DataLength [in]	Data length of the received message.
pData [in]	Pointer to the data of the received message (read only).
Return code	
Can_ReturnType	CAN_OK Higher layer indication will be called afterwards (CanIf_RxIndication()).
Can_ReturnType	CAN_NOT_OK Higher layer indication will not be called afterwards.
Functional Description	
Common RX indication callback that will be called before message specific callback will be called.	
Particularities and Limitations	
- Application callback function which informs about all incoming RX messages including the contained data. It can be used to block notification to upper layer. E.g. to filter incoming messages or route it for special handling. Configuration Variant(s): CAN_GENERIC_PRECOPY == STD_ON "pData" is read only and must not be accessed for further write operations. The parameter DataLength refers to the received data length by the CAN controller hardware. Note, that the CAN protocol allows the usage of data length values greater than eight (CAN-FD). Depending on the implementation of this callback it may be necessary to consider this special case (e.g. if the data length is used as index value in a buffer write access). None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-29 Appl_GenericPrecopy

6.2.30 Appl_GenericConfirmation

Prototype	
`Can_ReturnType` **`Appl_GenericConfirmation`** `(PduIdType PduId)`	
Parameter	
PduId [in]	Handle of the PDU specifying the message.
Return code	
Can_ReturnType	CAN_OK Higher layer confirmation will be called afterwards (CanIf_TxConfirmation()).
Can_ReturnType	CAN_NOT_OK Higher layer confirmation will not be called afterwards.
Functional Description	
Common TX notification callback that will be called before message specific callback will be called.	
Particularities and Limitations	
-	
Application callback function which informs about TX messages being sent to the CAN bus. It can be used to block confirmation or route the information to other layer as well.	
Configuration Variant(s): CAN_GENERIC_CONFIRMATION == STD_ON "PduId" is read only and must not be accessed for further write operations.	
None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-30 Appl_GenericConfirmation

6.2.31 Appl_GenericConfirmation

Prototype	
`Can_ReturnType` **`Appl_GenericConfirmation`** `(uint8 Controller, Can_PduInfoPtrType DataPtr)`	
Parameter	
Controller [in]	CAN controller which send the message.
DataPtr [in]	Pointer to a Can_PduType structure including ID (include IDE,FD), DataLength, PDU and data pointer.
Return code	
Can_ReturnType	CAN_OK Higher layer (CanInterface) confirmation will be called.
Can_ReturnType	CAN_NOT_OK No further higher layer (CanInterface) confirmation will be called.
Functional Description	
Common TX notification callback that will be called before message specific callback will be called.	
Particularities and Limitations	
-	
Application callback function which informs about TX messages being sent to the CAN bus. It can be used to block confirmation or route the information to other layer as well.	
Configuration Variant(s): CAN_GENERIC_CONFIRMATION == CAN_API2 A new transmission within this call out will corrupt the DataPtr context.	
None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-31 Appl_GenericConfirmation

6.2.32 Appl_GenericPreTransmit

Prototype	
`void Appl_GenericPreTransmit (uint8 Controller, Can_PduInfoPtrType_var DataPtr)`	
Parameter	
Controller [in]	CAN controller on which the message will be send.
DataPtr [in]	Pointer to a Can_PduType structure including ID (include IDE,FD), DataLength, PDU and data pointer.
Return code	
void	none
Functional Description	
Common transmit callback.	
Particularities and Limitations	
- Application callback function allowing the modification of the data to be transmitted (e.g.: add CRC). Configuration Variant(s): CAN_GENERIC_PRETRANSMIT == STD_ON None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-32 Appl_GenericPreTransmit

6.2.33 ApplCanTimerStart

Prototype	
`void ApplCanTimerStart (CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
void	none
Functional Description	
Start time out monitoring.	
Particularities and Limitations	
-	
Service function to start an observation timer.	
Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && ((CAN_HARDWARE_CANCELLATION == STD_ON) || MICROSAR4) Please refer to chapter "Hardware Loop Check".	
None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-33 ApplCanTimerStart

6.2.34 ApplCanTimerLoop

Prototype	
`Can_ReturnType ApplCanTimerLoop (CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
Can_ReturnType	> CAN_NOT_OK when loop shall be broken (observation stops) > CAN_NOT_OK should only be used in case of a time out occurs due to a hardware issue. > After this an appropriate error handling is needed (see chapter Hardware Loop Check / Time out Monitoring).
Can_ReturnType	> CAN_OK when loop shall be continued (observation continues)
Functional Description	
Time out monitoring.	
Particularities and Limitations	
- Service function to check (against generated maximum loop value) whether a hardware loop shall be continued or broken. Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && ((CAN_HARDWARE_CANCELLATION == STD_ON) || MICROSAR4) Please refer to chapter "Hardware Loop Check". None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-34 ApplCanTimerLoop

6.2.35 ApplCanTimerEnd

Prototype	
`void ApplCanTimerEnd (CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
void	none
Functional Description	
Stop time out monitoring.	
Particularities and Limitations	
-	
Service function to to end an observation timer.	
Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && ((CAN_HARDWARE_CANCELLATION == STD_ON) || MICROSAR4) Please refer to chapter "Hardware Loop Check".	
None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-35 ApplCanTimerEnd

6.2.36 ApplCanInterruptDisable

Prototype	
`void ApplCanInterruptDisable (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for the CAN interrupt lock.
Return code	
void	none
Functional Description	
CAN interrupt disabling by application.	
Particularities and Limitations	
- Disabling of CAN Interrupts by the application. E.g.: the CAN driver itself should not access the common Interrupt Controller due to application specific restrictions (like security level). Or the application like to be informed because of an CAN interrupt lock. called by Can_DisableControllerInterrupts(). Configuration Variant(s): CAN_INTLOCK == CAN_APPL || CAN_INTLOCK == CAN_BOTH None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-36 ApplCanInterruptDisable

6.2.37 ApplCanInterruptRestore

Prototype	
`void ApplCanInterruptRestore (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for the CAN interrupt unlock.
Return code	
void	none
Functional Description	
CAN interrupt restore by application.	
Particularities and Limitations	
- Re-enabling of CAN Interrupts by the application. E.g.: the CAN driver itself should not access the common Interrupt Controller due to application specific restrictions (like security level). Or the application like to be informed because of an CAN interrupt lock. called by Can_EnableControllerInterrupts(). Configuration Variant(s): CAN_INTLOCK == CAN_APPL || CAN_INTLOCK == CAN_BOTH None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-37 ApplCanInterruptRestore

6.2.38 Appl_CanOverrun

Prototype	
`void Appl_CanOverrun (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for which the overrun was detected.
Return code	
void	none
Functional Description	
Overrun detection.	
Particularities and Limitations	
- Called when an overrun is detected for a BasicCAN mailbox. Alternatively a DET call can be selected instead of ("CanOverrunNotification" is set to "DET"). Configuration Variant(s): CAN_OVERRUN_NOTIFICATION == CAN_APPL None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-38 Appl_CanOverrun

6.2.39 Appl_CanFullCanOverrun

Prototype	
`void Appl_CanFullCanOverrun (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for which the overrun was detected.
Return code	
void	none
Functional Description	
Overrun detection.	
Particularities and Limitations	
- Called when an overrun is detected for a FullCAN mailbox. Alternatively a DET call can be selected instead of ("CanOverrunNotification" is set to "DET"). Configuration Variant(s): CAN_OVERRUN_NOTIFICATION == CAN_APPL None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-39 Appl_CanFullCanOverrun

6.2.40 Appl_CanCorruptMailbox

Prototype	
`void Appl_CanCorruptMailbox (uint8 Controller, Can_HwHandleType hwObjHandle)`	
Parameter	
Controller [in]	CAN controller for which the check failed.
hwObjHandle [in]	Hardware handle of the defect mailbox.
Return code	
void	none
Functional Description	
Mailbox notification in case of RAM check failure.	
Particularities and Limitations	
- Will notify the application (during Can_InitController()) about a defect mailbox within the CAN cell. Configuration Variant(s): CAN_RAM_CHECK == CAN_NOTIFY_MAILBOX None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-40 Appl_CanCorruptMailbox

6.2.41 Appl_CanRamCheckFailed

Prototype	
`uint8 Appl_CanRamCheckFailed (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for which the check failed.
Return code	
uint8	CAN_DEACTIVATE_CONTROLLER deactivate the controller
uint8	CAN_ACTIVATE_CONTROLLER activate the controller
Functional Description	
CAN controller notification in case of RAM check failure.	
Particularities and Limitations	
- will notify the application (during Can_InitController()) about a defect CAN controller due to a previous failed mailbox check. The return value decide how to proceed with the initialization. Configuration Variant(s): CAN_RAM_CHECK == CAN_NOTIFY_MAILBOX / CAN_NOTIFY_ISSUE None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-41 Appl_CanRamCheckFailed

6.2.42 ApplCanReadProtectedRegister

Prototype	
`vuint16 ApplCanReadProtectedRegister (volatile vuint16 *addr)`	
Parameter	
addr [in]	Address of the register which has to be read
Return code	
vuint16	Returns the value of the register that was read
Functional Description	
Callback to read 16bit of protected register.	
Particularities and Limitations	
-	
Application function which is called by the CAN driver to read 16bit of special protected register when CAN driver runs in user mode.	
Configuration Variant(s): C_ENABLE_USER_MODE_APPL (defined via user configuration file) None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-42 ApplCanReadProtectedRegister

6.2.43 ApplCanReadProtectedRegister32

Prototype	
`vuint32 ApplCanReadProtectedRegister32 (volatile vuint32 *addr)`	
Parameter	
addr [in]	Address of the register which has to be read
Return code	
vuint32	Returns the value of the register that was read
Functional Description	
Callback to read 32bit of protected register.	
Particularities and Limitations	
-	
Application function which is called by the CAN driver to read 32bit of special protected register when CAN driver runs in user mode.	
Configuration Variant(s): C_ENABLE_USER_MODE_APPL (defined via user configuration file) None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-43 ApplCanReadProtectedRegister32

6.2.44 ApplCanWriteProtectedRegister

Prototype	
`void ApplCanWriteProtectedRegister (volatile vuint16 *addr, vuint16 mask, vuint16 val)`	
Parameter	
addr [in]	Address of the register which has to be written
mask [in]	The mask specifies the bits of the register to be written
val [in]	The value which has to be written to the register
Return code	
void	Returns the value of the register that was read
Functional Description	
Callback to write 16bit of protected register.	
Particularities and Limitations	
-	
Application function which is called by the CAN driver to write 16bit of special protected register when CAN driver runs in user mode.	
Configuration Variant(s): C_ENABLE_USER_MODE_APPL (defined via user configuration file) None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-44 ApplCanWriteProtectedRegister

6.2.45 ApplCanWriteProtectedRegister32

Prototype	
`void ApplCanWriteProtectedRegister32 (volatile vuint32 *addr, vuint32 mask, vuint32 val)`	
Parameter	
addr [in]	Address of the register which has to be written
mask [in]	The mask specifies the bits of the register to be written
val [in]	The value which has to be written to the register
Return code	
void	Returns the value of the register that was read
Functional Description	
Callback to write 32bit of protected register.	
Particularities and Limitations	
-	
Application function which is called by the CAN driver to write 32bit of special protected register when CAN driver runs in user mode.	
Configuration Variant(s): C_ENABLE_USER_MODE_APPL (defined via user configuration file) None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 6-45 ApplCanWriteProtectedRegister32

6.2.46 ApplCanPowerOnGetBaseAddress

Prototype	
`void* ApplCanPowerOnGetBaseAddress (uint32 physAddr, uint16 size)`	
Parameter	
physAddr [in]	The physical address (controller base address) to be converted into a virtual address
size [in]	The size of the memory block that is mapped to virtual memory
Return code	
void*	Returns the virtual address to be used by the driver
Functional Description	
Callback function to set the virtual address in the driver during Can_Init().	
Particularities and Limitations	
-	
Application function which is called by the CAN driver during Can_Init() to set the corresponding virtual address of a physical address (CAN controller hardware base address).	
Configuration Variant(s): C_ENABLE_UPDATE_BASE_ADDRESS (defined via user configuration file) None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	
Example Code	

```
void* ApplCanPowerOnGetBaseAddress(uint32 physAddr, uint16 size)
{
  void * virtAddr;
 .....
  if(physAddr == 0xE6E80000UL) /* example CAN controller physical base address 0 */
  {
    virtAddr = (void *) 0x06C30000UL; /* example CAN controller virtual address 0 provided by OS */
  }
  if(physAddr == 0xE6C38000UL) /* example CAN controller physical base address 1 */
  {
    virtAddr = (void *) 0x06C38000UL; /* example CAN controller virtual address 1 provided by OS */
  }
  …
  if(physAddr == ….) /* example CAN controller physical base address n */
  {
    virtAddr = …; /* example CAN controller virtual address n provided by OS */
  }
```

```
  …
  return virtAddr;
}
```

Table 6-46 ApplCanPowerOnGetBaseAddress

6.3 Services used by CAN

In the following table services provided by other components, which are used by the CAN are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	**Det_ReportError** (see “Development Error Reporting”)
DEM	**Dem_ReportErrorStatus** (see “Production Code Error Reporting”)
EcuM	**EcuM_CheckWakeup** This function is called when Wakeup over CAN bus occur. **EcuM_GeneratorCompatibilityError** This function is called during the initialization, of the CAN Driver if the Generator Version Check or the CRC Check fails. (see [5])
Application (optional non AUTOSAR)	**Appl_GenericPrecopy** **Appl_GenericConfirmation** **Appl_GenericPreTransmit** **ApplCanTimerStart/Loop/End** **Appl_CanRamCheckFailed, Appl_CanCorruptMailbox** **ApplCanInterruptDisable/Restore** **Appl_CanOverrun, Appl_CanFullCanOverrun** For detailed description see Chapter 6.2
CANIF	**CanIf_CancelTxNotification** (non AUTOSAR) A special Software cancellation callback only used within Vector CAN driver CAN Interface bundle. **CanIf_TxConfirmation** Notification for a successful transmission. (see [4]) **CanIf_CancelTxConfirmation** Notification for a successful Tx cancellation. (see [4]) **CanIf_RxIndication** Notification for a message reception. (see [4]) **CanIf_ControllerBusOff** Bus Off notification function. (see [4]) **CanIf_ControllerModeIndication** MICROSAR4x only: Notification for mode successfully changed.
Os (MICROSAR4x)	**OS_TICKS2MS_<counterShortName>()**

Component	API
	Os macro to get time based ticks from counter. **GetElapsedValue** Get elapsed tick count. **GetCounterValue** Get tick count start.

Table 6-47 Services used by the CAN

7 Configuration

For CAN driver the attributes can be configured with configuration Tool “GENy”

The CAN driver supports pre-compile, link-time and post-build configuration.

For post-build loadable systems, re-flashing the generated data can change some configuration settings.

For post-build and link-time configurations pre-compile settings are configured at compile time and therefore unchangeable at link or post-build time.

The following parameters are set by GENy configuration (see Chapter “Configuration with GENy”).

7.1 Pre-Compile Parameters

Some settings have to be available before compilation:

- Version API (Can_GetVersionInfo() activation)
  #define CAN_VERSION_INFO_API STD_ON/STD_OFF
- DET (development error detection)
  #define CAN_DEV_ERROR_DETECT STD_ON/STD_OFF
- Hardware Loop Check (timeout monitoring)
  #define CAN_HARDWARE_CANCELLATION STD_ON/STD_OFF
- Polling modes: Tx confirmation, Reception, Wakeup, BusOff
  #define CAN_TX_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_RX_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_BUSOFF_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_WAKEUP_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_INDIVIDUAL_PROCESSING STD_ON/STD_OFF
- Multiplexed Tx (external PIA – by use multiple Tx mailboxes)
  #define CAN_MULTIPLEXED_TRANSMISSION STD_ON/STD_OFF
- Configuration Variant (define the configuration type when using post-build variant)
  #define CAN_ENABLE_SELECTABLE_PB
- Use Generic Precopy Function (None AUTOSAR feature)
  #define CAN_GENERIC_PRECOPY STD_ON/STD_OFF
- Use Generic Confirmation Function (None AUTOSAR feature)
  #define CAN_GENERIC_CONFIRMATION STD_ON/STD_OFF
- Use Rx Queue Function (None AUTOSAR feature)
  #define CAN_RX_QUEUE STD_ON/STD_OFF
- Used ID type (standard/extended or mixed ID format)
  #define CAN_EXTENDED_ID STD_ON/STD_OFF
  #define CAN_MIXED_ID STD_ON/STD_OFF
- Usage of Rx and Tx Full and BasicCAN objects (deactivate only when not using and to save ROM and runtime consumption)

#define CAN_RX_FULLCAN_OBJECTS STD_ON/STD_OFF
#define CAN_TX_FULLCAN_OBJECTS STD_ON/STD_OFF
#define CAN_RX_BASICCAN_OBJECTS STD_ON/STD_OFF

- Use Multiple BasicCAN Rx objects
  #define CAN_MULTIPLE_BASICCAN STD_ON/STD_OFF
- Use Multiple BasicCAN Tx objects
  #define CAN_MULTIPLE_BASICCAN_TX STD_ON/STD_OFF
- Optimizations
  #define CAN_ONE_CONTROLLER_OPTIMIZATION STD_ON/STD_OFF
  #define CAN_DYNAMIC_FULLCAN_ID STD_ON/STD_OFF
- Usage of nested CAN interrupts
  #define CAN_NESTED_INTERRUPTS STD_ON/STD_OFF
- Use Multiple ECU configurations
  #define CAN_MULTI_ECU_CONFIG STD_ON/STD_OFF
- Use RAM Check (verify mailbox buffers)
  #define CAN_RAM_CHECK x
- Use Overrun detection
  #define CAN_OVERRUN_NOTIFICATION x
- Select MicroSar version
  #define CAN_MICROSAR_VERSION
  CAN_MSR403/CAN_MSR40/CAN_MSR30
- Tx Cancellation of Identical IDs
  #define CAN_IDENTICAL_ID_CANCELLATION STD_ON/STD_OFF
- Workaround for ERR005829/e5641
  #define C_ENABLE/DISABLE_WORKAROUND_ERR005829
- TASD computation
  #define C_ENABLE/DISABLE_TASD

7.2 Link-Time Parameters

The library version of the CAN driver use following generated settings:

- Maximum amount of used controllers and Tx mailboxes (has to be set for post-build variants at linktime)
- Rx Queue size
- Controller mapping (mapping of logical channel to hardware node).

7.3 Post-Build Parameters

Following settings are post-build data that can be changed for re-flashing:

- Amount and usage of FullCAN Rx and Tx mailboxes
- Used database (message information like ID, DLC)

- Filters for BasicCAN Rx mailbox
- Baud-rate settings
- Module Start Address (only for post-build loadable systems: The memory location for re-flashed data has to be defined)
- Configuration ID (only for post-build loadable systems: This number is used to identify the post-build data

7.4 Configuration with GENy

The CAN driver is configured with the help of the configuration tool GENy.

To generate communication specific settings, like used message objects e.g. FullCAN Rx mailboxes, baud rate settings and hardware specific settings for CAN communication, the GENy tool is used. This chapter explains the usage of this tool.

Please refer to the online help of the tool for detailed information about the general usage. And see below for special settings of the hardware specifics.

7.4.1 Platform Settings

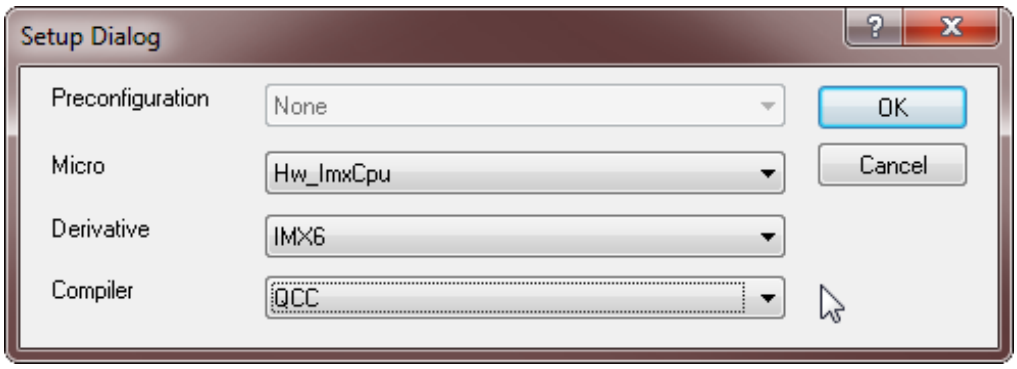


Figure 7-1 Platform settings

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
Micro	pre-compile	Enum	Hw_ImxCpu Hw_VybridCpu Hw_S32Cpu	Select the hardware platform to use
Derivative	pre-compile	Enum	IMX6X, IMX6, VF532R, S32K144, […]	The specific derivative of the platform
Compiler	pre-compile	Enum	GNU, GHS, QCC, […]	Compiler selection

Table 7-1 Platform Parameter description

7.4.2 Component Settings

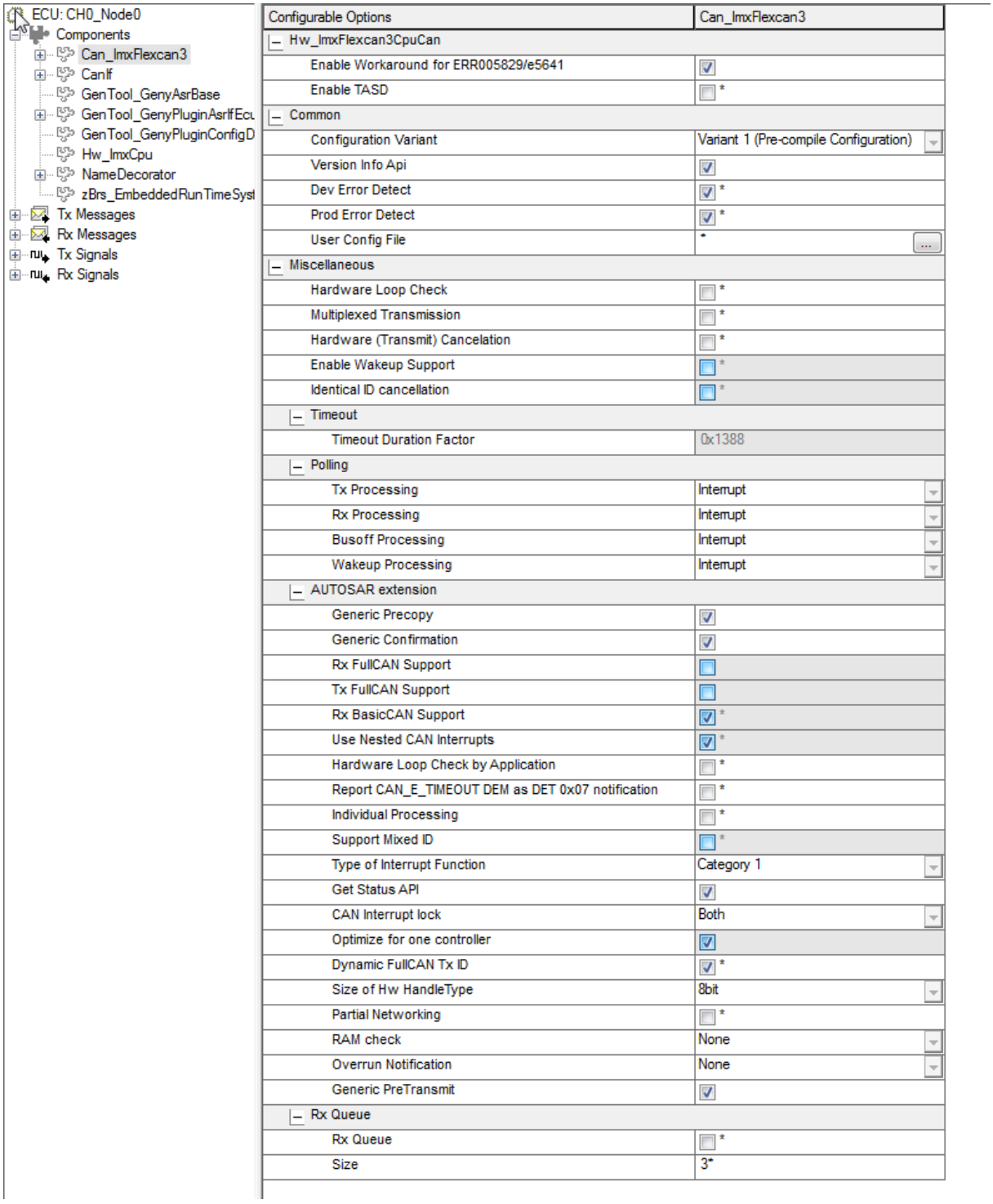


Table 7-2 Component Settings

Attribute Name	Configuration Variant	Value Type	Values	Description
Hw_ImxFlexcan3CpuCan				

Attribute Name	Configuration Variant	Value Type	Values	Description
Enable Workaround for ERR005829/e5641	pre-compile	Bool	On / Off	If this workaround is enabled one mailbox is reserved by the tool and cannot be used for communication. See errata sheet of semiconductor manufacturer for detailed information.
Enable TASD	pre-compile	Bool	On / Off	If this feature is enabled, the internal arbitration delay is computed to achieve an optimal arbitration start point depending on the hardware layout and bus timing configuration. If this feature stays disabled the default value 16 is used. TASD = Tx Arbitration Start Delay. See hardware manual for further information.
Miscellaneous				
Hardware Loop Check	pre-compile	Bool	On / Off	Timeout monitoring to check for hardware problems inside a loop (possible endless loops because of hardware issues will be notified over DEM)
Timeout Duration Factor (MICROSAR3 only)	pre-compile	Integer	Loop count	Maximum loop count for “Hardware Loop Check”. Refer to loop descriptions set this value for your needs.
Timeout Duration (MICROSAR4x only)	pre-compile	Float	Time	Specifies the time (in seconds) for the “Hardware Loop Check”.
Timeout Counter ID (MICROSAR401 only)	pre-compile	Float	Time	Specifies the counter ID used in OS for 'Timeout Duration'
Multiplexed Transmission	pre-compile	Bool	On / Off	Activate to get multiplexed transmit objects for Tx BasicCAN object (3 send objects instead of 1) avoids external Id priority inversion
Wakeup Support	pre-compile	Bool	On / Off	Activate to Wake up over CAN bus by CAN driver
Polling				
Tx Processing	pre-compile	Enum	Interrupt / Polling	Activate to handle transmit messages over polling

Attribute Name	Configuration Variant	Value Type	Values	Description
Rx Processing	pre-compile	Enum	Interrupt / Polling	Activate to handle receive messages over polling
Busoff Processing	pre-compile	Enum	Interrupt / Polling	Activate to handle Bus Off event over polling
Wakeup Processing	pre-compile	Enum	Interrupt / Polling	Activate to handle Wakeup event over polling
Mainfunction mode period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_Mode() in seconds
Mainfunction bus off period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_BusOff() in seconds
Mainfunction wakeup period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_Wakeup() in seconds
Mainfunction read period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_Read() in seconds
Mainfunction write period (MICROSAR4 only)	pre-compile	Float	Time	Period to call Can_MainFunction_Write() in seconds
AUTOSAR extension				
Generic PreCopy	pre-compile	Bool	On / Off	Generic Rx notification for all messages
Generic Confirmation	pre-compile	Bool	On / Off	Generic Tx confirmation for all messages
Generic PreTransmit	pre-compile	Bool	On / Off	A generic PreTransmit, common for all Tx messages will be called when this checkbox is enabled. Use this to change data or abort transmission right before the message will be send.
Rx FullCAN Support	pre-compile	Bool	On / Off	Use Rx FullCAN message objects (deactivate to reduce ROM and runtime consumption)
Tx FullCAN Support	pre-compile	Bool	On / Off	Use Tx FullCAN message objects (deactivate to reduce ROM and runtime consumption)
Rx BasicCAN Support	pre-compile	Bool	On / Off	Use Rx BasicCAN message objects (deactivate to reduce ROM and runtime consumption)
Use Nested CAN Interrupts	pre-compile	Bool	On / Off	CAN ISRs from different controllers can interrupt each other (only with higher priority). deactivate to reduce runtime consumption.

Attribute Name	Configuration Variant	Value Type	Values	Description
Secure Rx Buffer used	pre-compile	Bool	On / Off	Temporary buffer for received data will be located in a unused mailbox object. This is to secure receive data by not locate it to common RAM.
Hardware Loop Check by Application	pre-compile	Bool	On / Off	Enable this when you like to handle “Hardware Loop Check” by your own. Using API functions “ApplCanTimerStart()“, „ApplCanTimerEnd()“ and „ApplCanTimerLoop()“
Report CAN_E_TIMEOUT DEM as DET 0x07 notification	pre-compile	Bool	On / Off	Enable this when you like to report CAN_E_TIMEOUT (“Hardware Loop Check”) to DET instead of DEM (does only work when DET is activated)
Individual Processing	pre-compile	Bool	On / Off	Enable this when you like to configure mailbox specific polling/interrupt processing. This feature is only available with “High End” license.
Support Mixed ID	pre-compile	Bool	On / Off	Force CAN driver to handle Mixed ID (standard and extended ID) at pre-compile-time to expand the ID type later on.
Multiple BasicCAN Objects	pre-compile	Bool	On / Off	Support Multiple BasicCAN objects. This gives the possibility to use multiple Filters and optimize acceptance Filtering as well as avoid overrun. This feature is only available with “High End” license.
Optimize for one controller	pre-compile	Bool	On / Off	Activate this for 1 controller systems when you never will expand to multi-controller. So that the CAN driver works more efficient
Dynamic FullCAN Tx ID	pre-compile	Bool	On / Off	Always write FullCAN Tx ID within Can_Write() API function. Deactivate this to optimize code when you do not use FullCAN Tx objects dynamically.
Size of Hw HandleType	pre-compile	Enum	8bit 16bit	Support 8bit Hardware Handles as define in AutoSar 3.01 specified. Or change to 16bit like specified in AutoSar 4.0.

Attribute Name	Configuration Variant	Value Type	Values	Description
Partial Networking	pre-compile	Bool	On / Off	Activate this when you like to support a partial network that will be woken up by special NM message. This feature have to be enabled if one of the controllers will wake up by partial network transceiver.
Get Status API	pre-compile	Bool	On / Off	Support Can_GetStatus() API that return the hardware states: CAN_STATUS_STOP, CAN_STATUS_INIT, CAN_STATUS_INCONSISTENT, CAN_STATUS_WARNING, CAN_STATUS_PASSIVE, CAN_STATUS_BUSOFF, CAN_STATUS_SLEEP
Type of Interrupt Function	Link-time	Enum	Category 1 Category 2 VoidFuncVoid	Category 1: Interrupt Function has to be added to the interrupt vector table unless this is not already done automatically by a compiler pragma. Category 2: Interrupt Function is defined with ISR() define. VoidFuncVoid: Interrupt Function is declared as void ISR(void) and has to be called in case of a CAN Interrupt.
CAN Interrupt lock	pre-compile	Enum	CANDriver Application Both	Disable and Restore Can Interrupt by CAN Driver or Application or CAN Driver and Application. Reason maybe, that the CAN driver should not access the Interrupt controller for CAN interrupts or there are other restrictions (special security level) or Application has to care about additional Interrupts like Wakeup.
Overrun Notification	pre-compile	Enum	None DET Application	Overrun detection activation and selection of the Notification function. None: No Overrun detection. DET: will be called for Overrun. Application: will be called.

Attribute Name	Configuration Variant	Value Type	Values	Description
RAM check	pre-compile	Enum	None Active Mailbox Notification	RAM check of mailbox buffers None: No RAM check Active: Active RAM check (global notification) Mailbox Notification: Active RAM check (global + mailbox notification)
Hardware (Transmit) Cancellation	pre-compile	Bool	On / Off	Enable/disable the usage of ‘transmit cancellation’ out of hardware mailboxes, so that the canceled message won’t be send. (avoid internal priority inversion)
Identical ID cancellation	pre-compile	Bool	On / Off	'Hardware Transmit Cancellation" also cancel messages with same identifier (newer data will be send) out of hardware mailboxes. otherwise only lower prior identifiers will be cancelled. (older data will be send)
Rx Queue				
Rx Queue	pre-compile	Bool	On / Off	Enable this when you like to handle Rx messages in polling context. (Shorten interrupt latency time). This feature is only available with “High End” license.
Size	pre-compile	Bool	On / Off	Size of FIFO in which the Message information is buffered.
Common				
Configuration Variant	pre-compile	Enum	Pre-Compile, Link-Time, Post build	Select your project type
Version Info API	pre-compile	Bool	On / Off	Activate function to get version information
Dev Error Detection	pre-compile	Bool	On / Off	Activate to get information about possible conflicts (refer to DET module description)
Prod Error Detection	pre-compile	Bool	On / Off	Activate to get information about possible conflicts in a running system (refer to DEM module description)
User Config File	pre-compile	String	File name and path	This file will be included in the Can_cfg.h for special settings
Post build loadable configuration				

Attribute Name	Configuration Variant	Value Type	Values	Description
Module Start Address (Post-build only)	post-build	Integer	Address	Address to Flash the data
Max Nr. Of Tx Objects (Post-build only)	Link-time	Integer	Amount	Amount of maximum used transmit objects over all controllers (used for RAM variables). One object for BasicCAN Tx and one for each FullCAN Tx. Add all objects over all used Channels for this value.
Max CAN Controller (Post-build only)	Link-time	Integer	Amount	Amount of maximum used controllers (used for RAM variables)
Configuration ID (Post-build only)	post-build	Integer	Identity	A special value to identify this configuration in the Flash.

Table 7-2 Component Parameter description

7.4.3 Individual Polling Settings

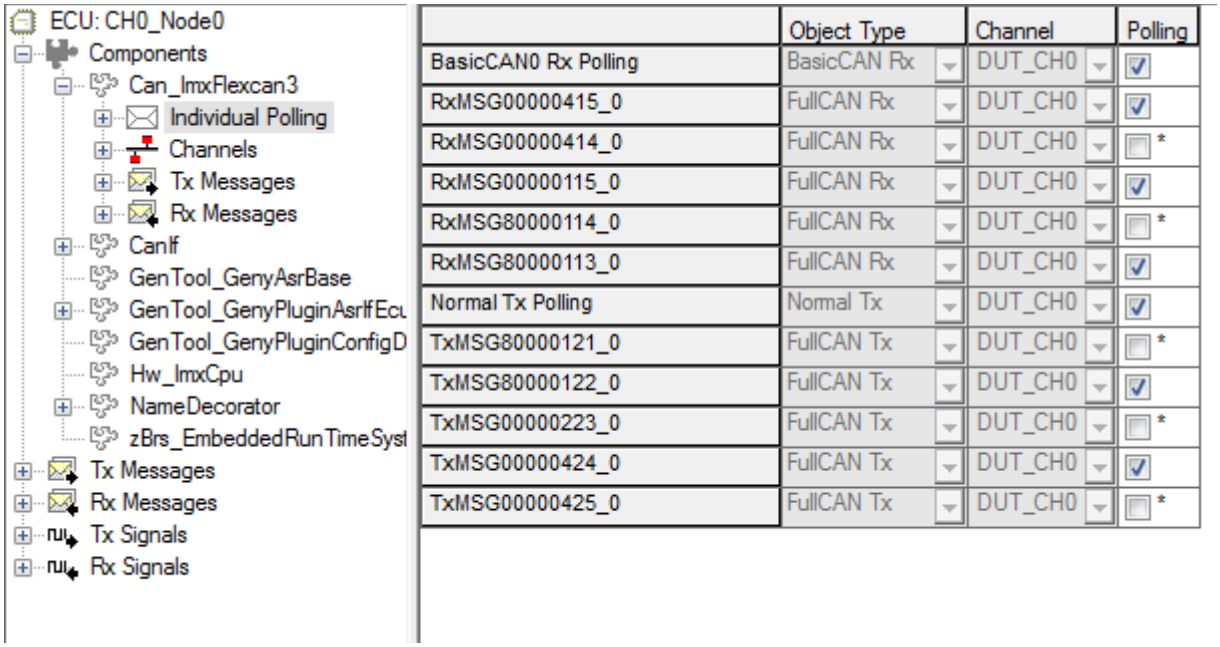


Figure 7-2 Individual polling

When “Individual Polling” feature in 7.4.2 is selected you can select for each mailbox Rx/Tx event to be polled or handled by interrupt. In case of FullCAN mailbox the name of the message is shown. In case of BasicCAN mailbox “BasicCAN Rx (x)” or “Normal Tx” has to be selected to handle all messages related to this mailbox in polling mode.

“Normal Tx” is the collection of all none FullCAN Tx messages.

“BasicCAN Rx (X)” is the collection of all none FullCAN Rx messages pass the BasicCAN Filter “X”.

7.4.4 Controller (Channel) Settings

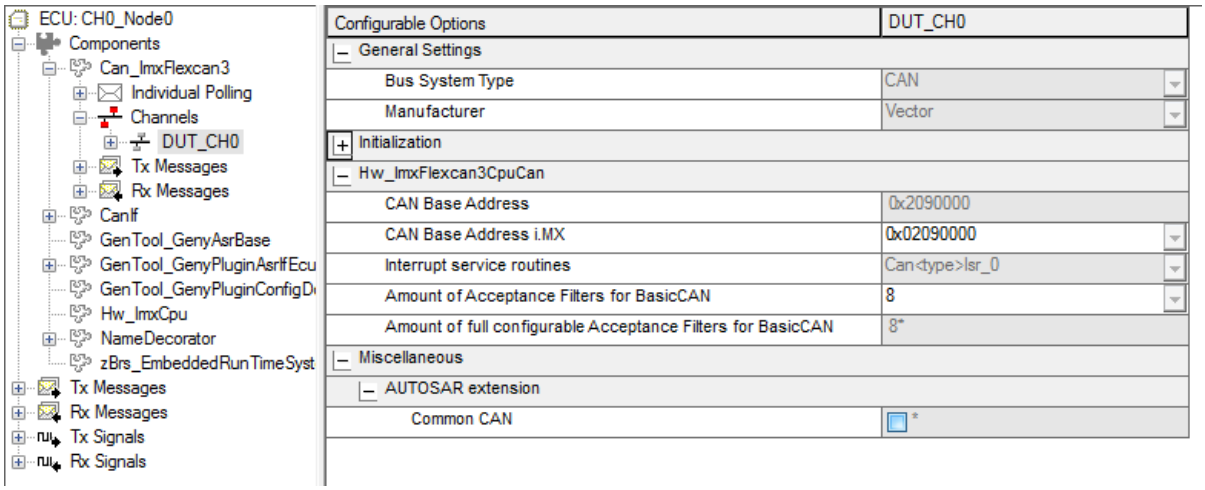


Figure 7-3 Controller Settings

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
Miscellaneous				
Wakeup Source ID / Wakeup Source Ref (Reference is only available if EcuM is available in GENy)	Link-time	Integer	Source	Insert the Wakeup Source value compatible to ECU State Manager.
Enable Wakeup Support	Link-time	Bool	On / Off	MICROSAR4x only: Activation of Wakeup over CAN bus.
AUTOSAR extension				
Partial Networking	Post-build	Bool	On / Off	Activate this when you there is a partial network transceiver on this controller active and used.
Common CAN	Link-time	Bool	On / Off	Activate this when you like to connect two physical CAN channels to one logical CAN Controller. This allows you to use more FullCAN for this Controller. Only the Rx FullCAN can be configured on second physical channel. RESTRICTIONS: 'CanCommonCAN' is a project specific feature and may is not

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
				available or supported for every platform.
CAN Base Address	Link-time	Enum	Addresses of supported CAN Controllers.	Memory mapped base addresses of the CAN Controllers. See hardware manual of the used derivative for detailed information.
Amount of AFs for BasicCAN	Post-build	Enum	Amount	Configure how many acceptance filters will be assigned to the Rx BasicCAN object (RxFifo) for this channel. Restriction: Some filters are not full configurable. These filters cannot be configured in the 'Acceptance Filter Configuration' dialog. They cannot be configured manually but are only considered from the Acceptance Filter Optimization process.
Amount of full configurable AFs for BasicCAN	Post-build	Integer	Automatically computed amount.	This value is calculated automatically if the amount of AFs for the Rx BasicCAN object is changed. This value matches the available filters in the 'Acceptance Filter Configuration' dialog.

Table 7-3 Controller Parameter description

7.4.5 Init Structure Settings

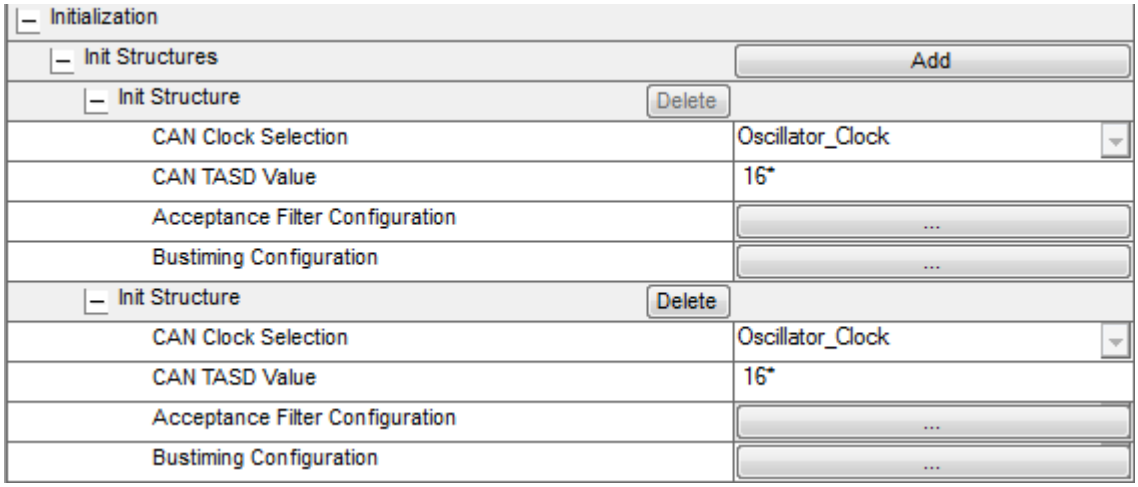


Figure 7-4 Init Structure Dialog

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values The default value is written in bold	Description
Initialization				
CAN Clock Selection	link-time, post-build	Enum	“Oscillator Clock” “System Clock”	Select the clock source of CAN unit.
CAN TASD Value	link-time, post-build	Integer	[0, 31] Depending on the derivative the range can be limited by a special maximum value. See hardware manual for further information.	Tx Arbitration start delay value can be used to delay the start point of the arbitration process to optimize transmission performance.
Acceptance Filter Configuration	link-time, post-build	Integer	See chapter 7.4.5.1	Acceptance filter configuration for BasicCAN Rx objects.
Bustiming Configuration	link-time, post-build	Integer	See chapter 7.4.5.2	Bit timing configuration

Add Init Structures to setup multiple baud rate and filter settings that can be selected at Initialization phase of the ECU.

MicroSar3 only: This structure can be selected with `Can_InitStruct()` before call `Can_InitController()`.

7.4.5.1 Setup the Filter by pressing the “Acceptance Filter Configuration” Button.

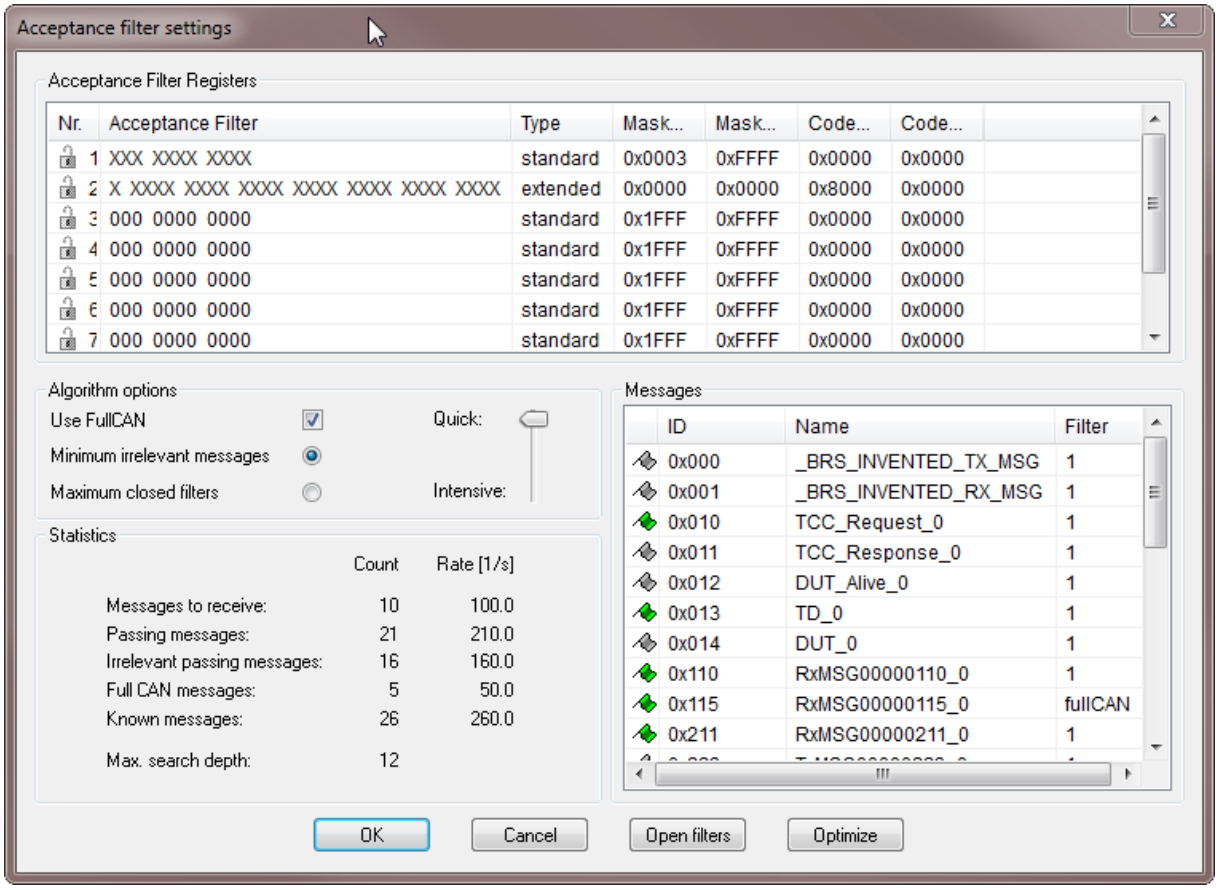


Figure 7-5 Setup Filter Dialog

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values The default value is written in bold	Description
Setup				
Acceptance Filter	Post-build	Integer	Mask, code	Each ID –bit is represented by a “0/1/X”, means must match a “0” or “1” or must not match “X”
Open filters		Action	-	Open the filter complete to receive all IDs
Optimize		Action	-	Open the filter automatic to just receive the IDs in database. “Use FullCAN” try to put as much messages in FullCAN objects to better optimize filters
Messages-Window			Messages that will be received via ranges (like NM messages) will have always a grey flag in view.	Open the filter automatic to just receive the IDs in database. “Use FullCAN” try to put as much messages in

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values The default value is written in bold	Description
			They will be included in “Irrelevant passing messages” amount.	FullCAN objects to better optimize filters

Table 7-4 Filter Parameter description

7.4.5.2 Setup the Baud rate by pressing the “Bus timing Configuration” Button.

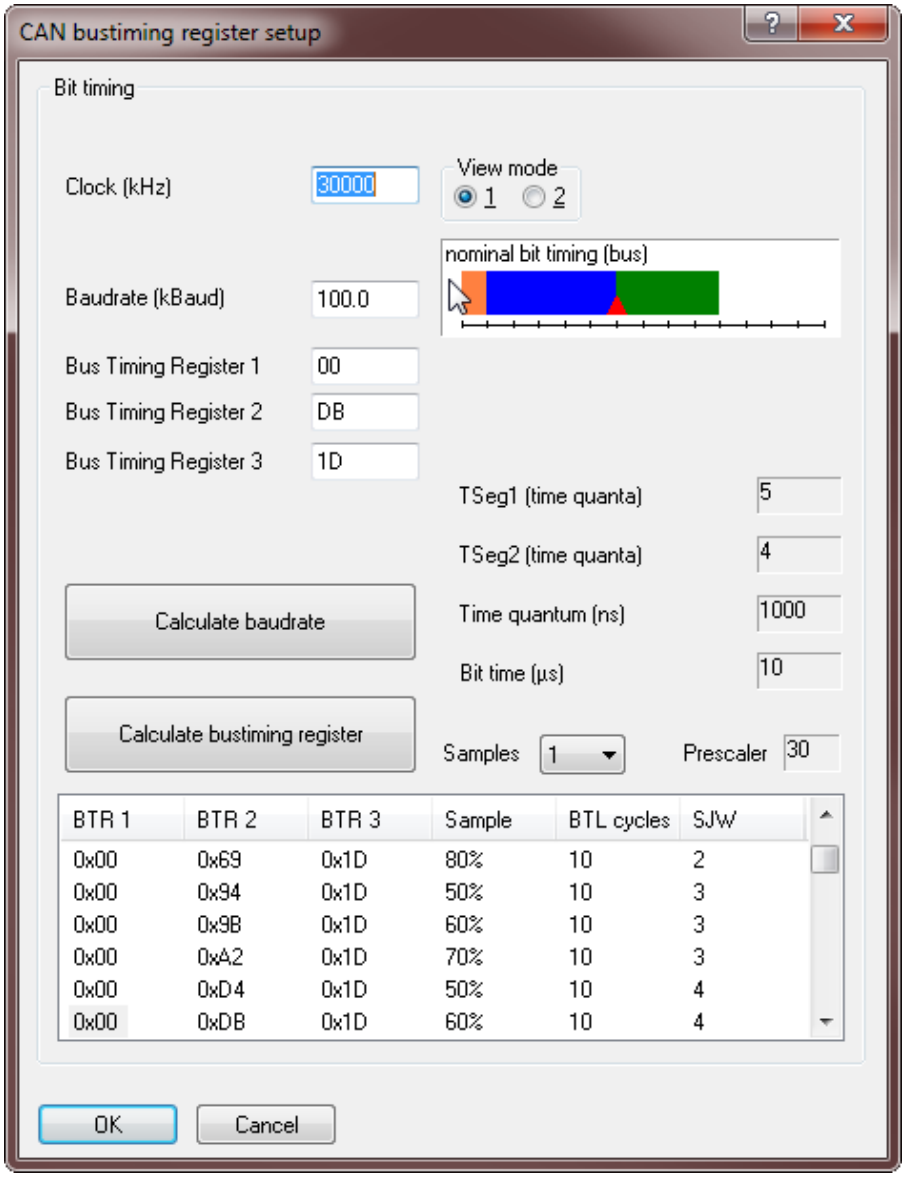


Figure 7-6 Baud Rate Dialog

Attribute Name	Attribute is of type (Configuration Variant)	Value Type	Values	Description
Clock	Post-build	Integer	CAN clock	Set the CAN cell clock.
Baudrate	Post-build	Integer	Baud rate	Set baud rate to be used for this channel
Calculate		Action	-	It is possible to calculate possible hardware register settings out of baud rate or vice versa.
Sample, BTL cycles, SJW	Post-build	Select	Sample Point	Select the sample point and sync phase related to your bus physics

Table 7-5 Baud rate Parameter description

7.5 Configuration with da DaVinci Configurator

See Online help within DaVinci Configurator and BSWMD file for parameter settings.

8 AUTOSAR Standard Compliance

8.1 Limitations / Restrictions

Category	Description	Version
Functional	No multiple AUTOSAR CAN driver allowed in the system	3.0.6
Functional	No support for L-PDU callout (AUTOSAR 3.2.1), but support ‘Generic Precopy’ instead	3.2.1
Functional	No support for multiple read and write period configuration	3.2.1
API	“Symbolic Name Values” may change their values after precompile phase so do not use it for Linktime or Postbuild variants. It’s recommended that higher layer generator use Values (ObjectIDs) from EcuC file. Vector CAN Interface does so.	3.0.6

8.2 Erratum ERR005829/e5641

FlexCAN: FlexCAN does not transmit a message that is enabled to be transmitted in a specific moment during the arbitration process.

See errata sheet of semiconductor manufacturer for detailed information and if the used derivative is concerned by this CAN Controller malfunction. How to enable the workaround is described in 7.4.2. Please note that for the workaround implementation one mailbox is declared as reserved by the generation tool and cannot be used for communication.

8.3 Vector Extensions

Refer to Chapter “Features” listed under “**AUTOSAR extensions”**

9 Glossary and Abbreviations

9.1 Glossary

Term	Description
GENy	Generation tool for CANbedded and MICROSAR components
High End (license)	Product license to support an extended feature set (see Feature table)

Table 9-1 Glossary

9.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution) 3,3x = AUTOSAR version 3 401 = AUTOSAR version 4.0.1 403 = AUTOSAR version 4.0.3 4x = AUTOSAR version 4.x.x
SWS	Software Specification
Common CAN	Connect two physical peripheral channels to one CAN bus (to increase the amount of FullCAN)
Hardware Loop Check	Timeout monitoring for possible endless loops.

Table 9-2 Abbreviations

10 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com